



WPI

CS 534

Artificial Intelligence

Week 8: Deep Learning II

By

Ben C.K. Ngan

Different Types of Deep Learning Architectures

- Convolutional Neural Network
- Recurrent Neural Network
- Autoencoders
- And More

DS 541. DEEP LEARNING

Convolutional Neural Network

- Convolutional Neural Network (CNN), also known as ConvNet.
- Computer Vision Applications (e.g., Image Processing, Video Processing, and Text Recognition)
- Non-image Data Application (e.g., Time Series Classification, <https://www.intechopen.com/chapters/64216>)

Chapter

CNN Approaches for Time Series classification

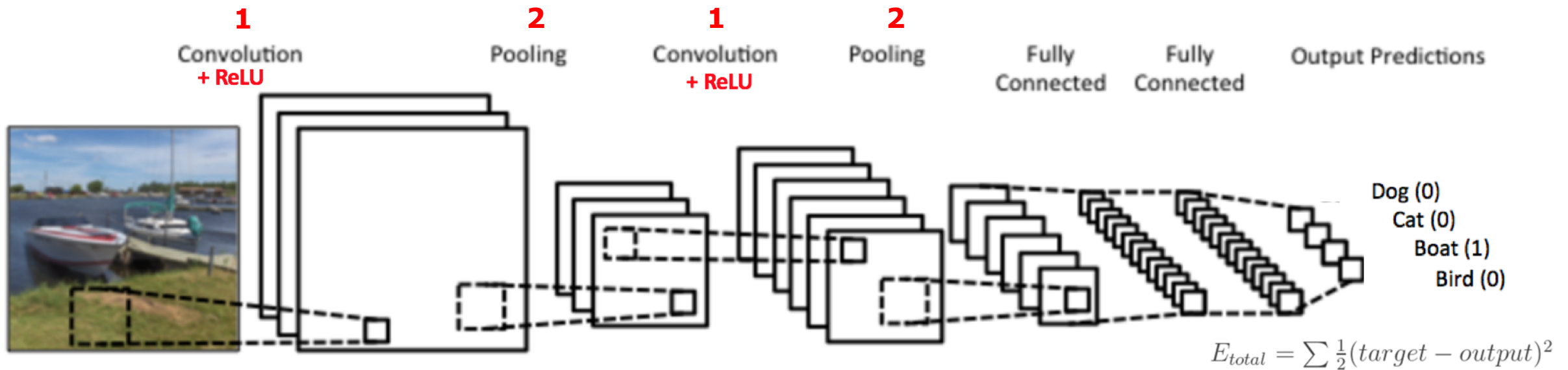
Lamyaa Sadouk

Abstract

Time series classification is an important field in time series data-mining which have covered broad applications so far. Although it has attracted great interests during last decades, it remains a challenging task and falls short of efficiency due to the nature of its data: high dimensionality, large in data size and updating continuously. With the advent of deep learning, new methods have been developed, especially Convolutional Neural Network (CNN) models. In this paper, we present a review of our time series CNN approaches including: (i) a data-level approach based on encoding time series into frequency-domain signals via the Stockwell transform, (ii) an algorithm-level approach based on an adaptive convolutional layer filter that suits the time series in hand, and (iii) another algorithm-level approach adapted to time series classification tasks with limited annotated data, which is a global, fast and light-weight framework based on a transfer learning technique with a source learning task similar or different but related to the target learning task. These approaches are implemented on identifying human activities including normal movements of typical subjects and disorder-related movements such as stereotypical motor movements of autistic subjects. Experimental results show that our approaches improve performance of time series classification.

Convolutional Neural Network

(Input -> Convolution -> ReLU -> Max pooling)
now add up a fully connected layer to the output

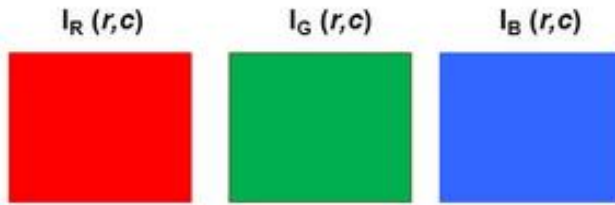


Convolution Operation
extracts simple patterns
(e.g., horizontal or vertical
lines or edges) in different
parts of the picture

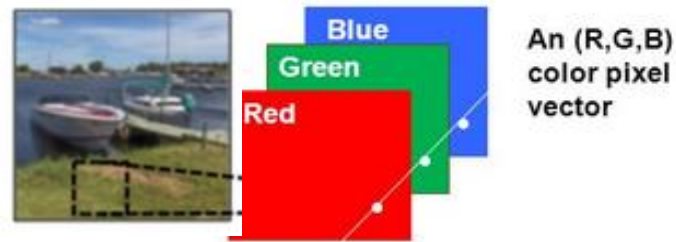
Feature Extraction from Image

Classification

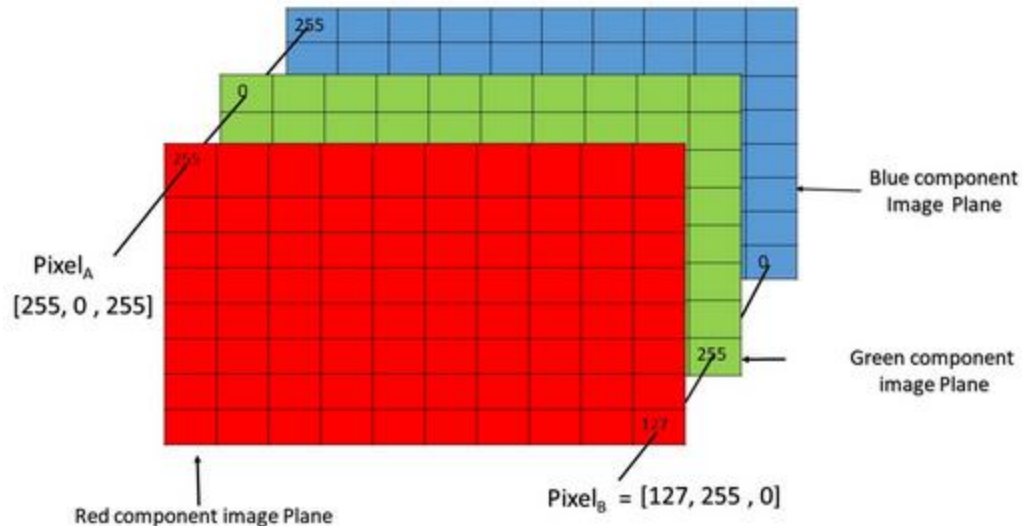
Convolutional Neural Network



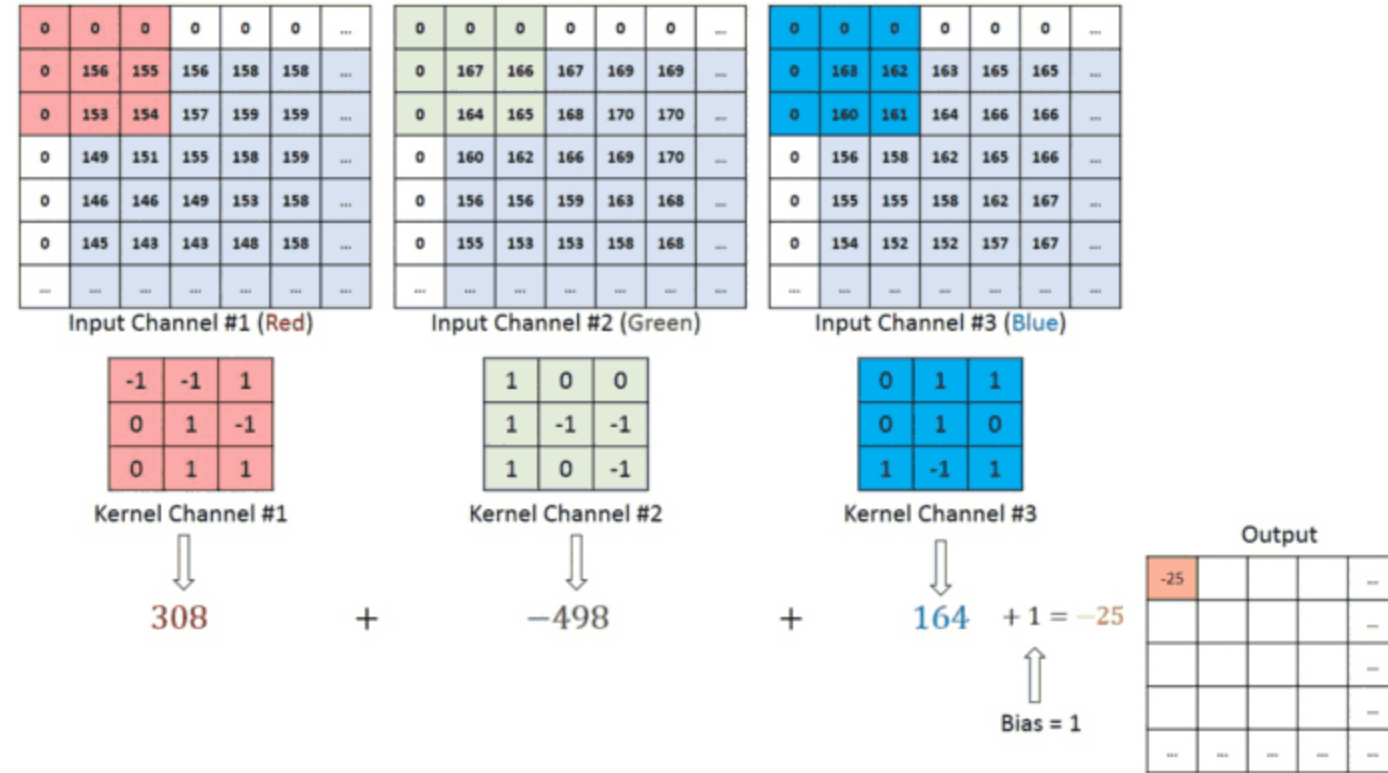
a. A typical RGB color image can be thought of as three separate images ($I_R(r,c)$, $I_G(r,c)$ and $I_B(r,c)$)



b. A color pixel vector consists of the red, green and blue pixel values (R,G,B) at one given row/column pixel coordinate (r,c)



Pixel of an RGB image are formed from the corresponding pixel of the three component images



RGB image value range: each color component can have is [0 – 255].

Convolutional Neural Network

Image (Red Component)

Input

4	9	2	5	8	3
6	2	4	0	3	
2	4	5	4	5	2
5	6	5	4	7	8
5	7	7	9	2	1
5	8	5	3	8	4

$n = 6$

$n_H \times n_W = 6 \times 6$

Filter

B (Feature Detector)

1	0	-1
1	0	-1
1	0	-1

$f = 3$

*

=

Result C (Feature Map)

2	6		

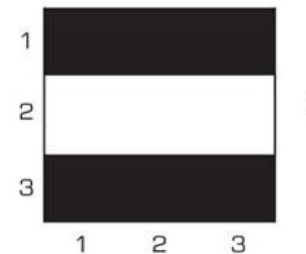
6 = $9 \cdot 1 + 2 \cdot 0 + 5 \cdot (-1) + 6 \cdot 1 + 2 \cdot 0 + 4 \cdot (-1) + 4 \cdot 1 + 5 \cdot 0 + 4 \cdot (-1)$

The resulting output matrix C will be smaller in dimension than A but larger than B.

Take a look the following three pixel map images as an example.

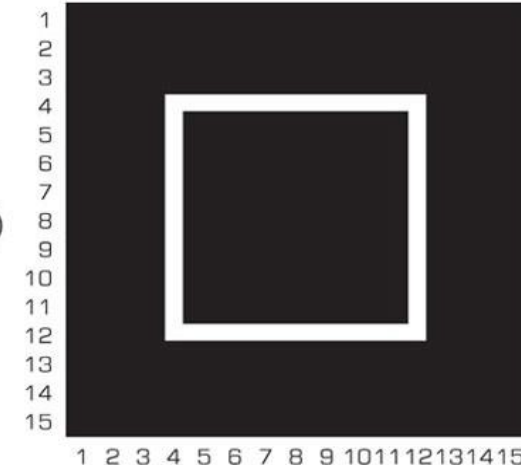
- Each cell is a pixel value.
- Image C highlights only certain features of the raw image A.

Convolution between two matrices, A and B, denoted as $A (*) B$, results in a new matrix C, whose elements are obtained by **a sum-product between individual elements of A and B.**



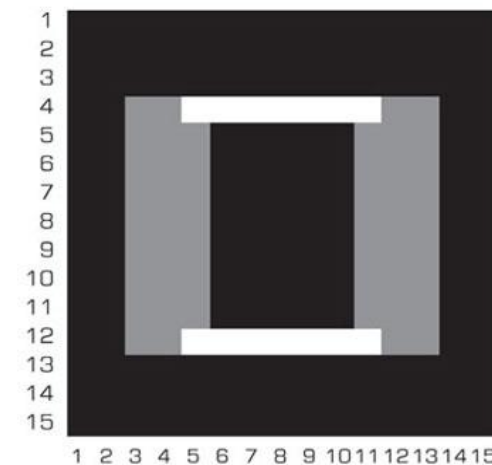
Horizontal Kernel

*



Input Image

=



Output Image

Convolutional Neural Network

Image (Red Component)

Input

4	9	2	5	8	3
6	2	4	0	3	
2	4	5	4	5	2
5	6	5	4	7	8
5	7	7	9	2	1
5	8	5	3	8	4

$n = 6$

$n_H \times n_W = 6 \times 6$

Filter

B (Feature Detector)

1	0	-1
1	0	-1
1	0	-1

$f = 3$

*

=

Result C (Feature Map)

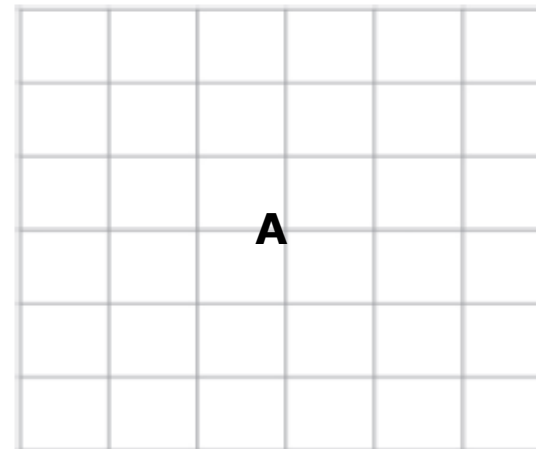
2	6		

$6 = 9 \cdot 1 + 2 \cdot 0 + 5 \cdot (-1) + 6 \cdot 1 + 2 \cdot 0 + 4 \cdot (-1) + 4 \cdot 1 + 5 \cdot 0 + 4 \cdot (-1)$

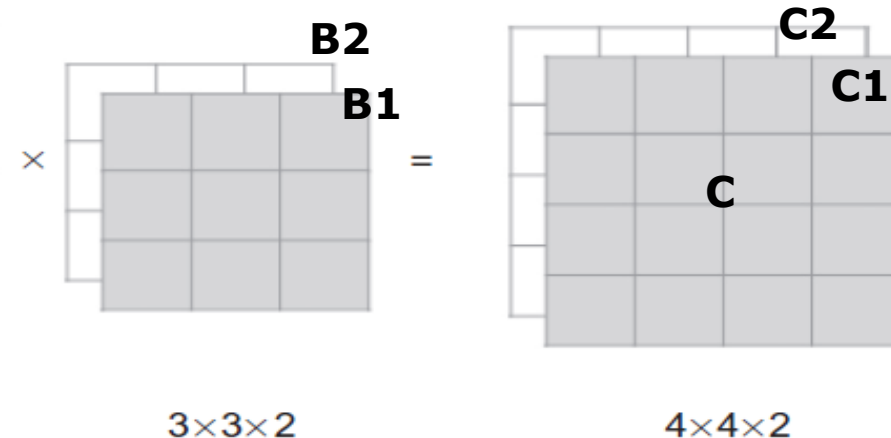
Multiple filters can be applied to the same image A.

- B1 filter be a horizontal edge detector,
- B2 filter be a vertical edge detector
- C can be represented as a volume of 3 dimensions, i.e., $4 \times 4 \times 2$, which is essentially a stack of two 4×4 matrices.
- Each one is the result of convolution between A and B_i , where $i = 1$ and 2 .

<https://indoml.com>



6×6

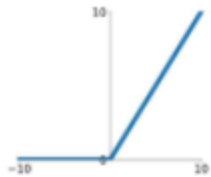


Convolution between two matrices, A and B, denoted as $A (*) B$, results in a new matrix C, whose elements are obtained by **a sum-product between individual elements of A and B.**

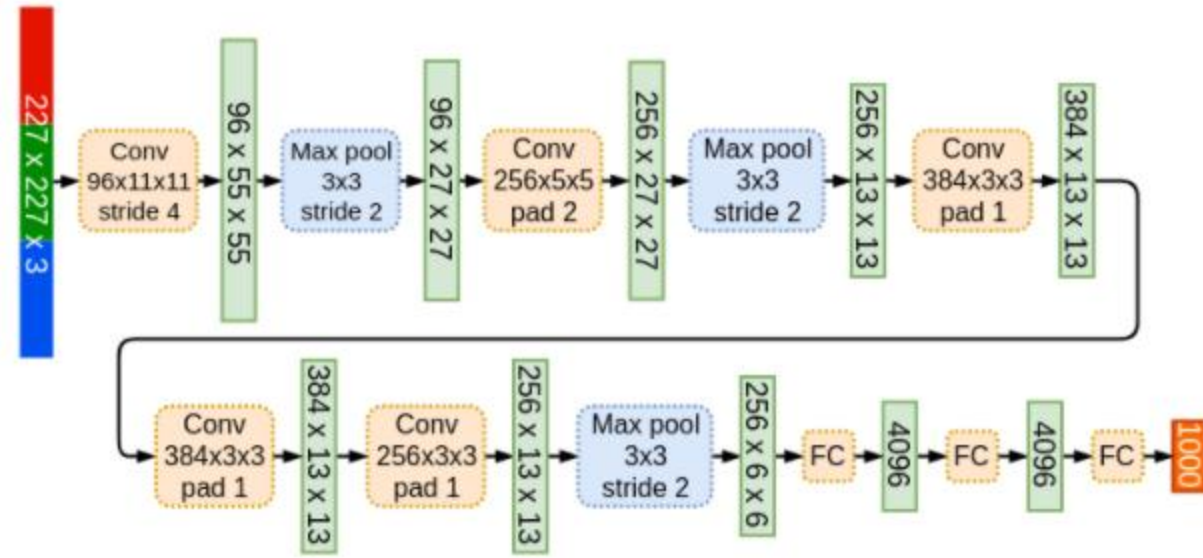
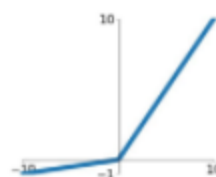
Convolutional Neural Network

- AlexNet Deep Learning System in the 2012 ImageNet Competition
- 1.2M Images in 1,000 Different Categories.
- Top-5 Error Rate: 26%

ReLU
 $\max(0, x)$



Leaky ReLU
 $\max(0.1x, x)$



	Hidden Layer	Design
Convolution	1	96 filters in size 11x11 with max pooling in size 3x3
	2	256 filters in size 5x5 with max pooling in size 3x3
	3	384 filters in size 3x3 without pooling in size 3x3
	4	384 filters in size 3x3 without pooling
	5	256 filters in size 3x3 with max pooling in size 3x3
Fully Connected	1	4096 nodes with LeakyRelu activation function
	2	4096 nodes with LeakyRelu activation function
	3	100 nodes with LeakyRelu activation function

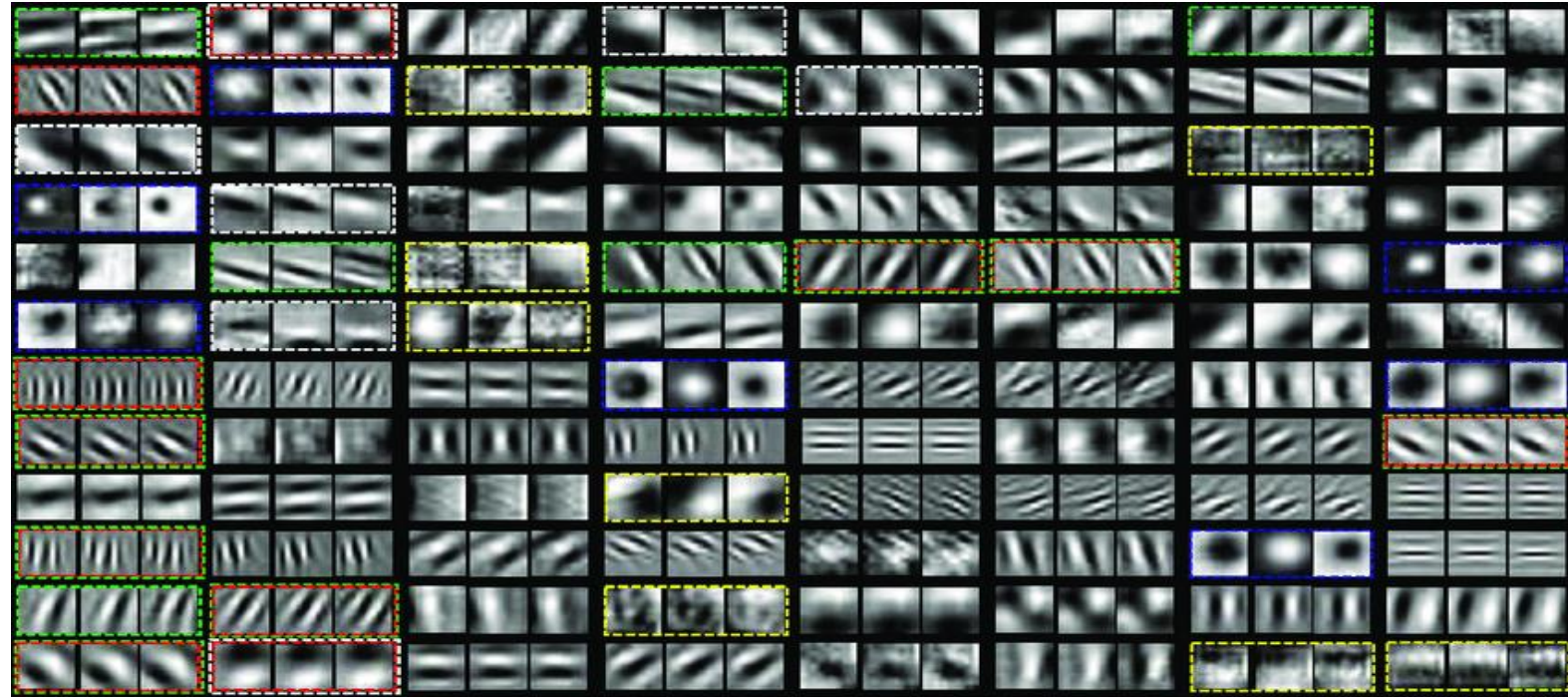
- In 2012, AlexNet significantly outperformed all the prior competitors and won the challenge by reducing the top-5 error from **26% to 15.3%**. The second place top-5 error rate was around **26.2%**.

Convolutional Neural Network

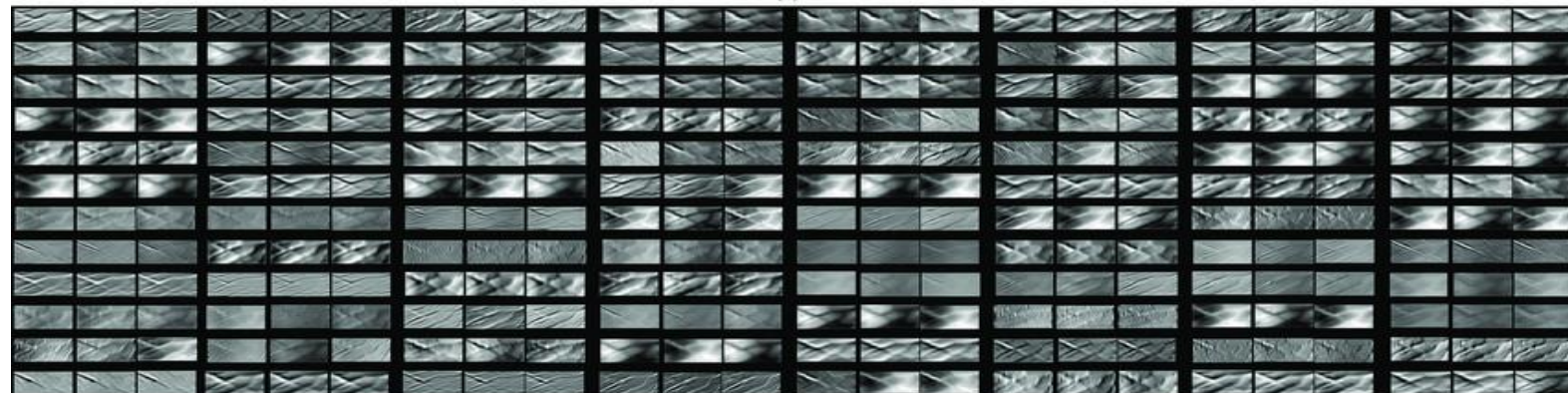
Visualizations of CNN filters and their outputs: **Alexnet** was very first successful network for image classification

(a) The 96 groups of CNN filters from the **first layer** of the AlexNet network

(b) CNN filtered images.



(a)

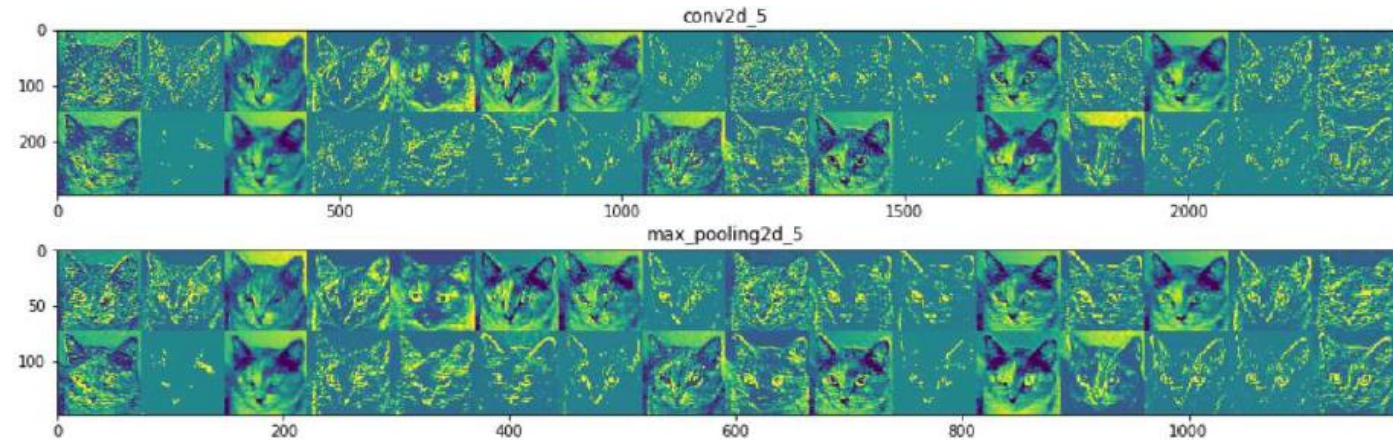


(b)

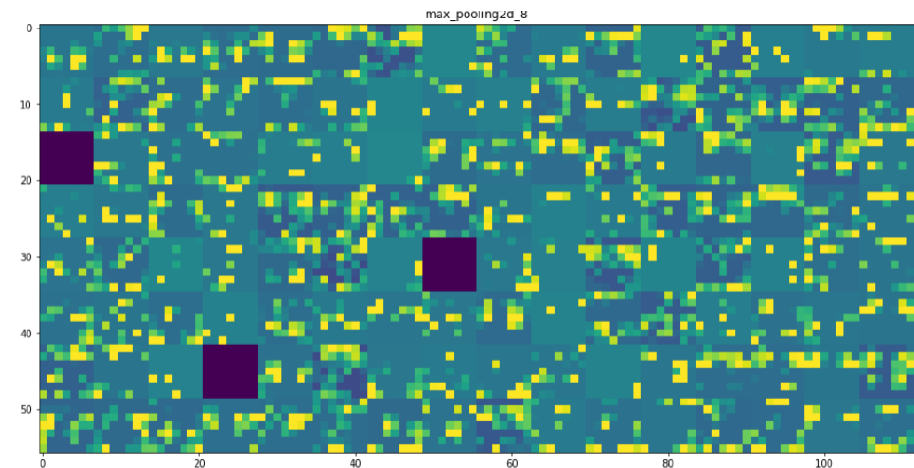
Convolutional Neural Network

Feature maps visualization

- **At the start**, the CNN and max pooling layers resemble the original image as they are edge detectors and **retaining almost all the information in initial picture**
- **As the layers start reaching the end**, only an **abstract representation** of cat remains (less visually interpretable) that is necessary to classify the image as a cat – **more information about target, less information about input**
- Neural networks are Information distillation pipeline



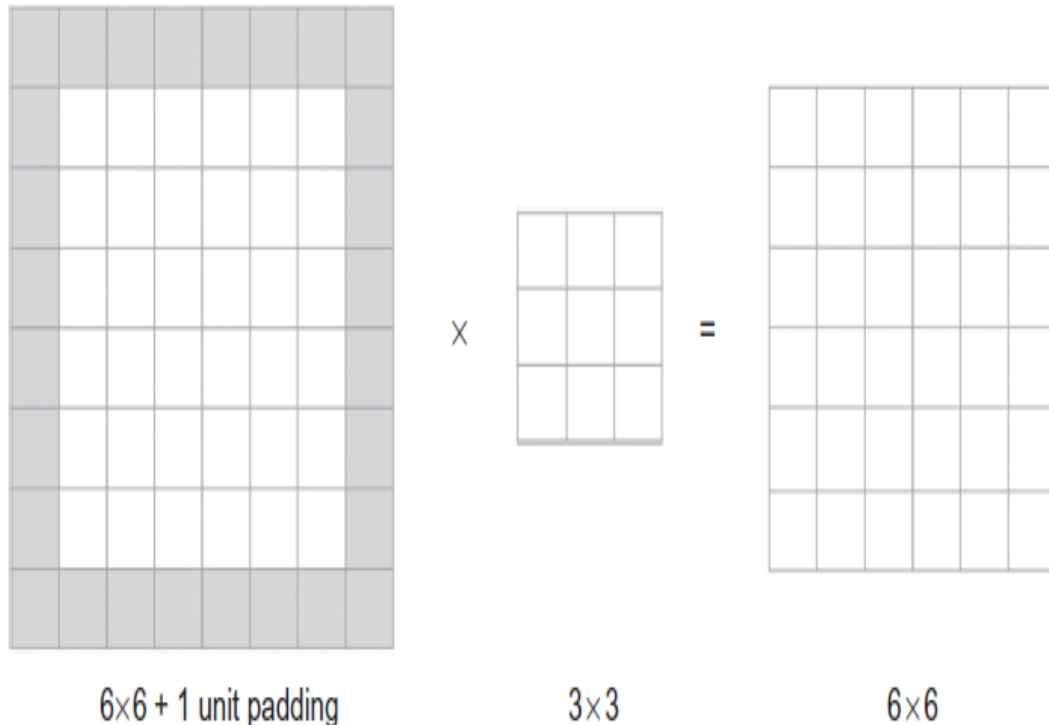
CNN Feature Map – Starting layers



CNN Feature Map – End layer

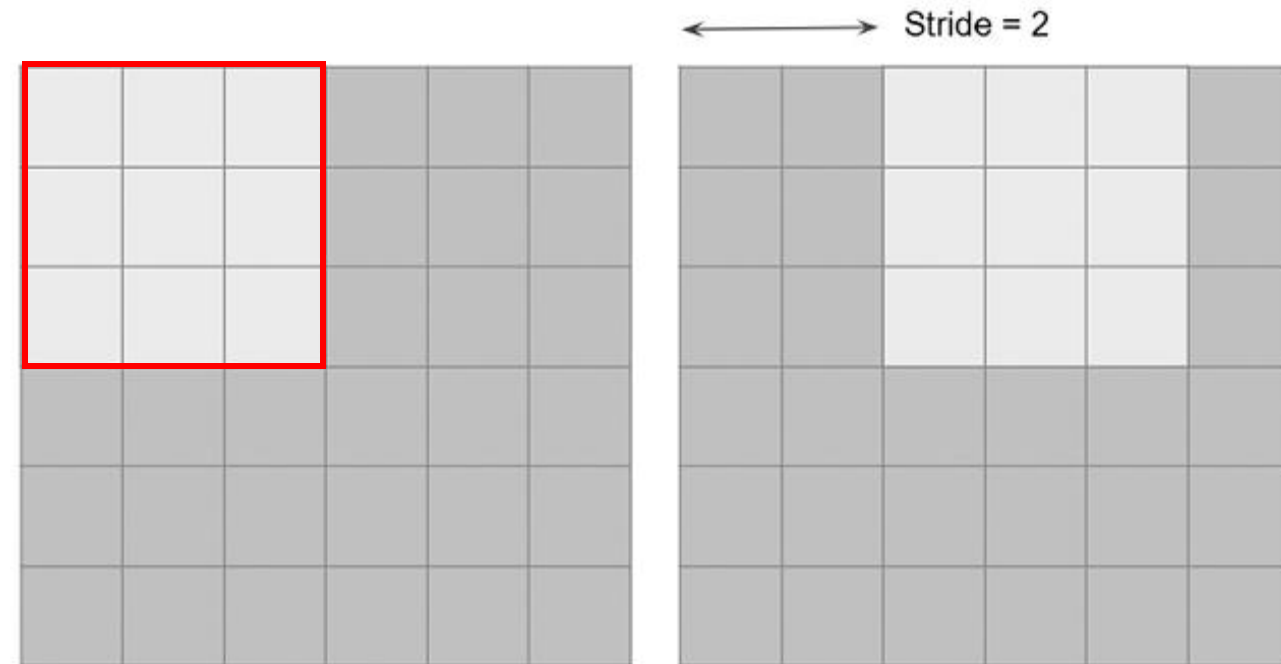
Convolutional Neural Network

Convolution with Padding (***p***): As the process of convolution reduces the raw image size, it is sometimes useful to **enlarge the raw image with dummy pixels** so that the original size is retained. This process is called "**padding**". padding means giving additional pixels at the boundary of the data.



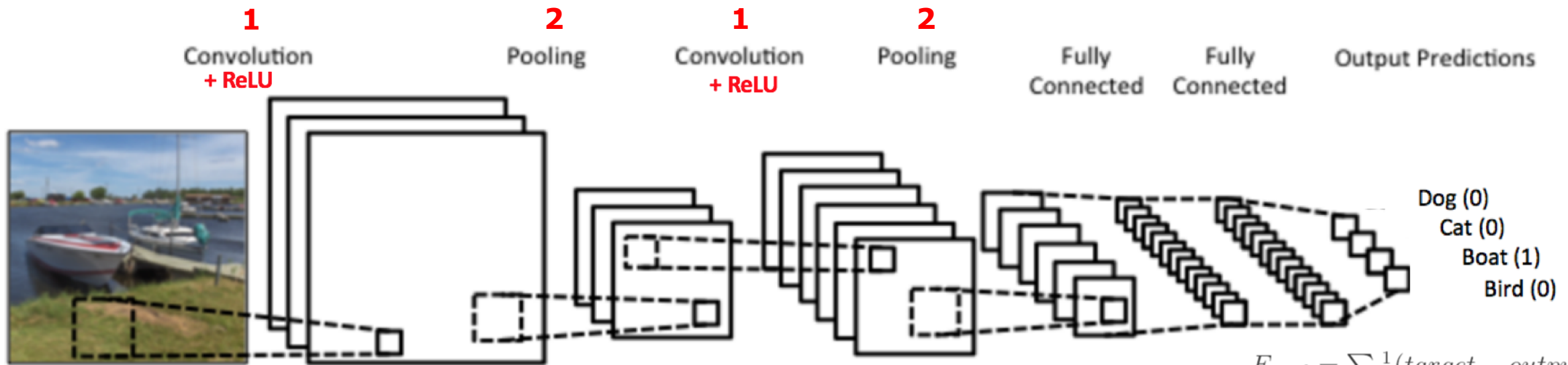
Convolution with Stride (***s***): This is the number of pixels the filter is advanced each time to compute the next element of the output.

By increasing ***s*** from 1 (default) to 2, the feature map size is reduced by a factor of 2.

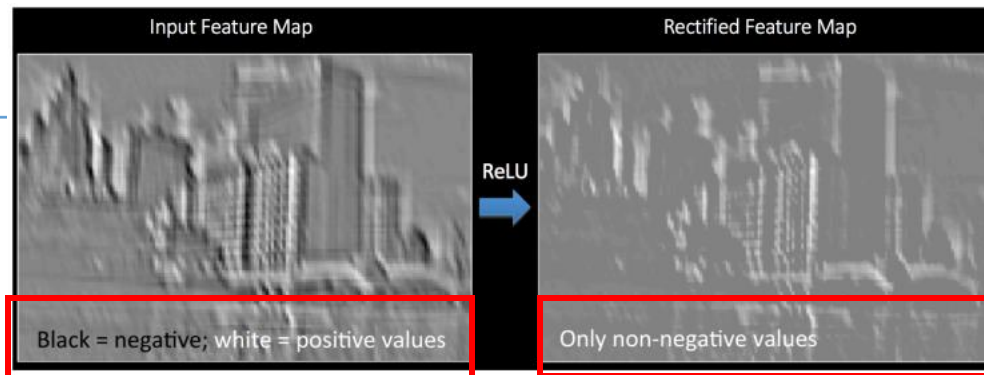


Convolutional Neural Network

(Input -> Convolution -> ReLU -> Max pooling)
now add up a fully connected layer to the output



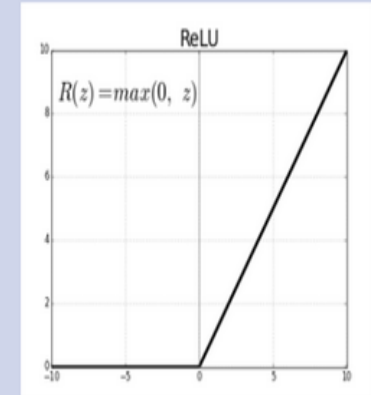
$$E_{total} = \sum \frac{1}{2} (target - output)^2$$



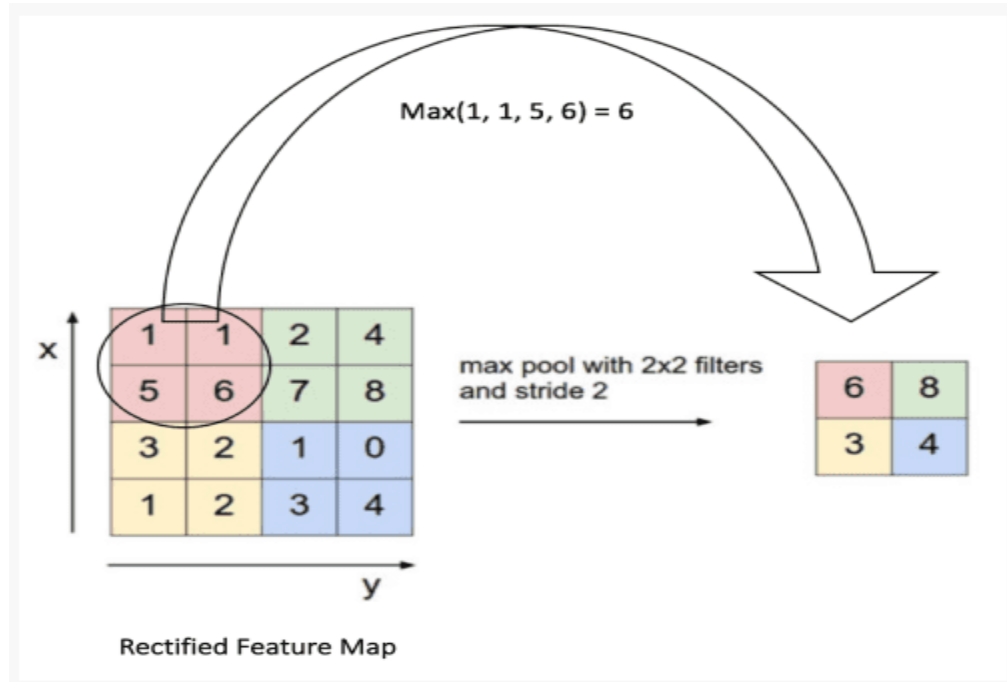
ReLU (Rectified Linear Unit)

$$R(z) = \max(0, z)$$

0 to ∞

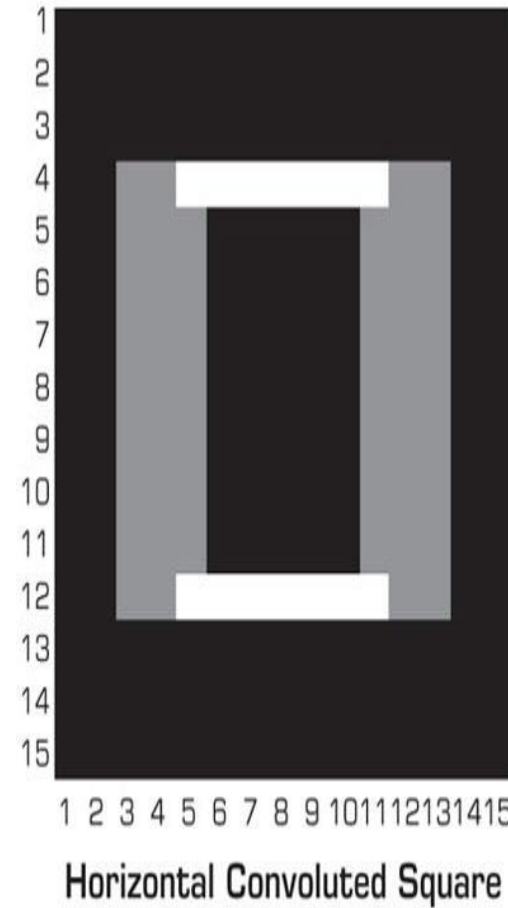


Convolutional Neural Network



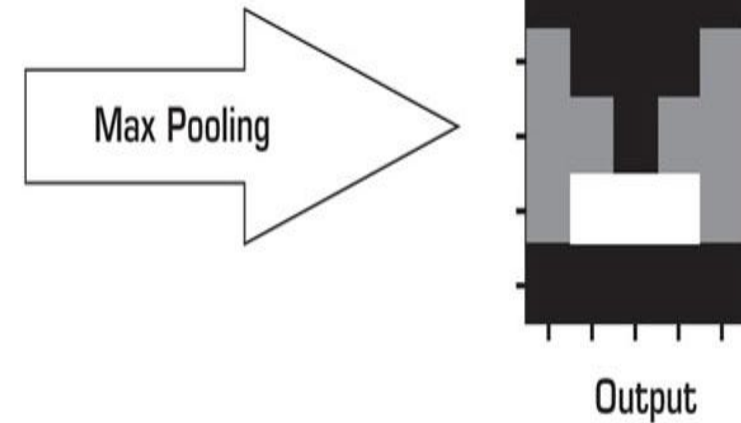
Average Pooling computes the average value to down sample it.

Max pooling can be thought of as an operation that highlights the most **dominant feature** in an image (e.g., an eye within a face). Max pooling is another feature detection tactic that is widely used for image processing.



Copyright © 2020 by Pearson Education, Inc.

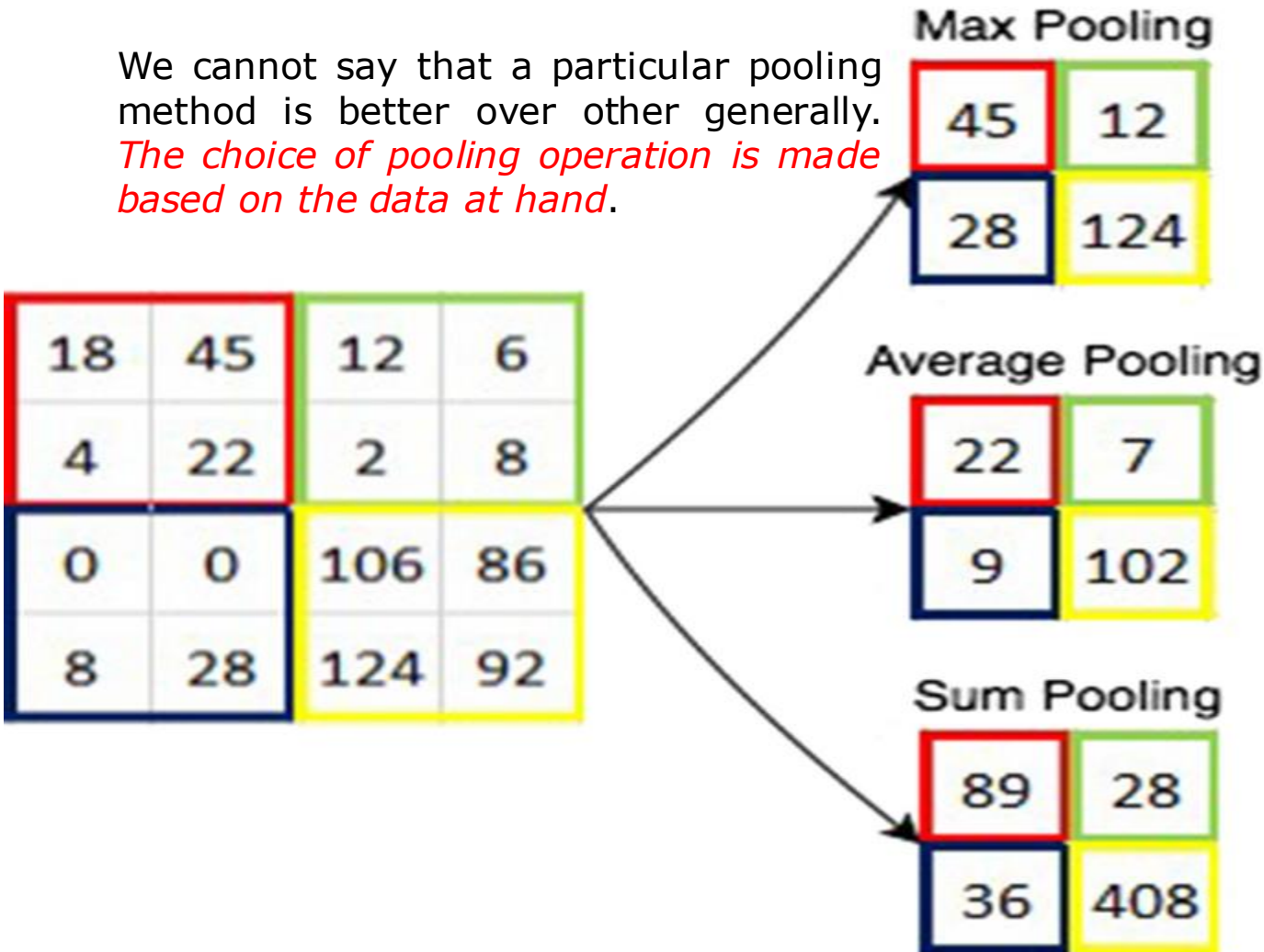
The pooling process can reduce the spatial size or dimension of each feature map through different formulations such as **max-pooling**, **min-pooling**, **average-pooling**, **sum-pooling**, etc.



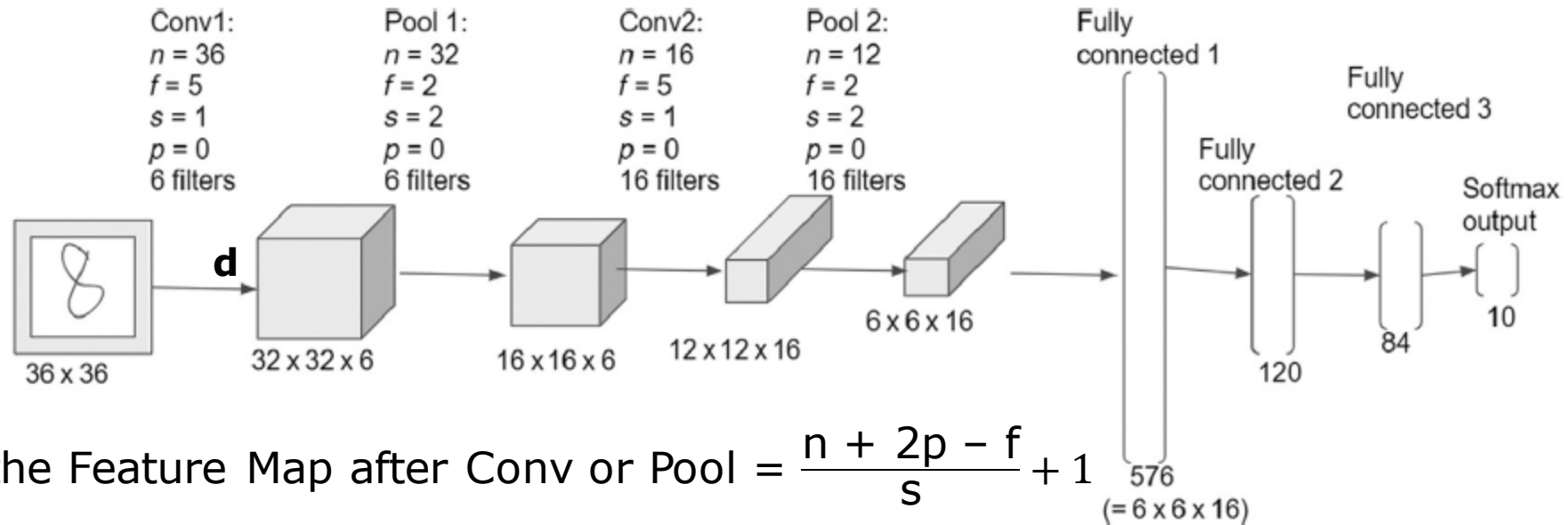
Convolutional Neural Network

The pooling process can reduce the spatial size or dimension of each feature map through different formulations such as **max-pooling**, **min-pooling**, **average-pooling**, **sum-pooling**, etc.

We cannot say that a particular pooling method is better over other generally.
The choice of pooling operation is made based on the data at hand.



Convolutional Neural Network



It is a class CNN architecture introduced by LeCun (1989) to recognize handwritten digits.

$n = n_{\text{width}} = n_{\text{height}}$ is the dimension of an image

$f = f_{\text{width}} = f_{\text{height}}$ is the dimension of a filter

s is the stride, the number of pixels that the filter is advanced each time to compute the next element of the output.

p is the number of padded pixels

Convolutional Neural Network

1	1	0
4	2	1
0	2	1

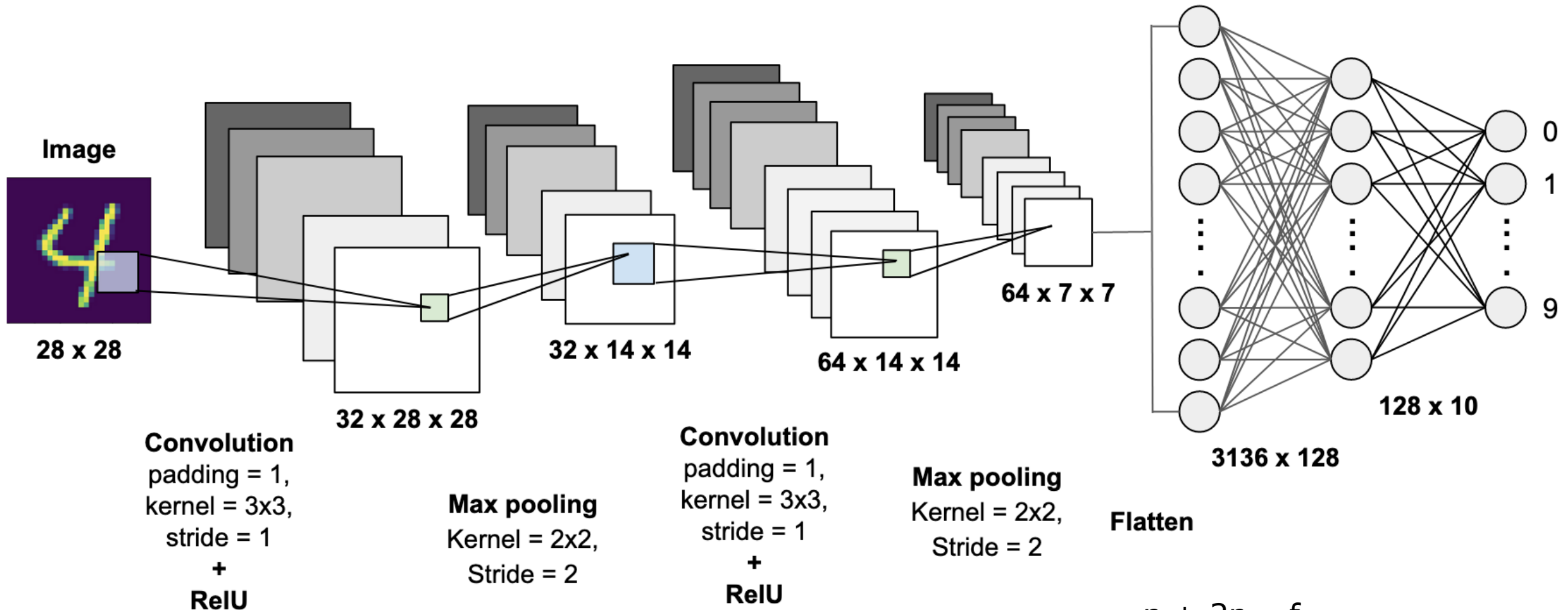
Pooled Feature Map

Flattening



1
1
0
4
2
1
0
2
1

Convolutional Neural Network



Dimension (d) of the Feature Map after Conv or Pool = $\frac{n + 2p - f}{s} + 1$

Choosing a Network Architecture

- Three Approaches to **Improve Network Performance**:
 - Choose a right network architecture.
 - Penalize large weights
 - Randomly perturb the values passing through the network during the training.
- Choose a network architecture
 - The deeper network gives a better performance
 - 11-layer network > 3-layer network.
- Neural Architecture Search
 - Genetic Algorithm
 - Hill Climbing
 - Reinforcement Learning

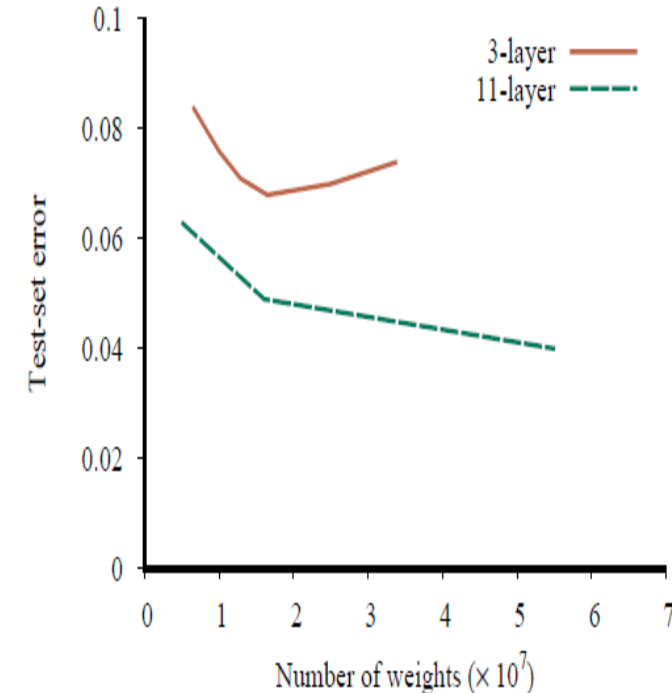


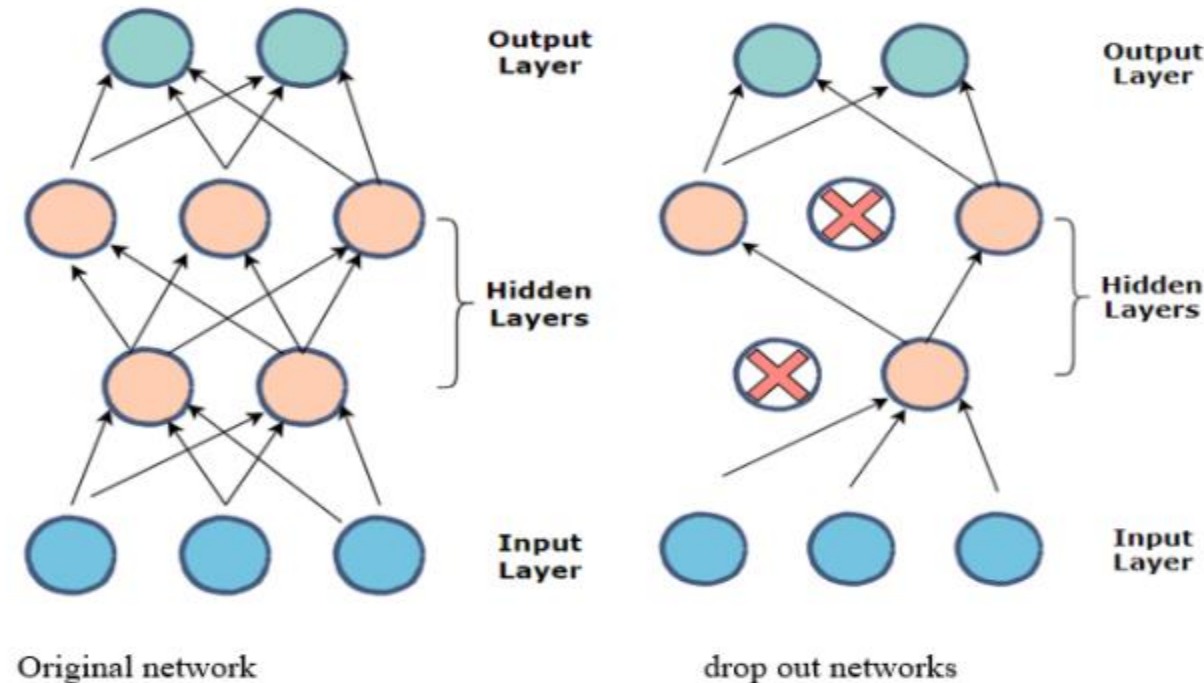
Figure 21.7 Test-set error as a function of layer width (as measured by total number of weights) for three-layer and eleven-layer convolutional networks. The data come from early versions of Google's system for transcribing addresses in photos taken by Street View cars

Penalizing Large Weights

- Large weights in a neural network are a sign of a more complex network that has **overfit** the training data.
- Penalizing a network based on the size of the network weights during training can **reduce overfitting**.

Randomly perturb the values passing through the network during the training - Dropout

- The term “dropout” refers to **dropping out units or neurons** (input, hidden and output) in a neural network for **reducing overfitting** and improving the performance.
- Specifically, dropout refers to ignoring units or neurons during the training phase of certain set of neurons which is chosen at random. By “ignoring”, **it means these units or neurons are not considered during a particular forward or backward pass.**



Hands-on Example: Convolution Neural Network

```
import tensorflow as tf
import matplotlib.pyplot as plt

def main():
    # Load training and testing data from the built-in dataset CIFAR.
    # The CIFAR-10 dataset consists of 60,000 32 x 32 color images in 10 classes,
    # with 6000 images per class. There are 50,000 training images and 10,000 test images

    (train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.cifar10.load_data()

    # Each of the images contains with 8-bit color information.
    # Let us normalize that information to a value between 0 and 1 by dividing it by 255.
    train_images, test_images = train_images / 255.0, test_images / 255.0

    # Also, we will initialize the class labels.
    class_names = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]

    # Let us take a look at our training data. The following loop will display the first 25 images with their corresponding labels.
    plt.figure(figsize=(10,10))
    for i in range(25):
        plt.subplot(5, 5, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        plt.imshow(train_images[i])

        # The CIFAR labels happen to be arrays
        # which is why you need the extra index
        plt.xlabel(class_names[train_labels[i][0]])
    plt.show()
```

<https://www.cs.toronto.edu/~kriz/cifar.html>

Hands-on Example: Convolution Neural Network



frog



truck



truck



deer



automobile



automobile



bird



horse



ship



cat



deer



horse



horse



bird



truck



truck



truck



cat



bird



frog



deer



cat



frog



frog



bird

Hands-on Example: Convolution Neural Network

```
# Build the CNN-based model with several multidimensional convolution layers
```

```
model = tf.keras.models.Sequential()  
model.add(tf.keras.layers.Conv2D(32, (3, 3), activation = "relu", input_shape=(32, 32, 3)))  
model.add(tf.keras.layers.MaxPooling2D((2, 2)))  
model.add(tf.keras.layers.Conv2D(64, (3, 3), activation = "relu"))  
model.add(tf.keras.layers.MaxPooling2D((2, 2)))  
model.add(tf.keras.layers.Conv2D(64, (3, 3), activation = "relu"))  
model.add(tf.keras.layers.Flatten())  
model.add(tf.keras.layers.Dense(64, activation = "relu"))  
model.add(tf.keras.layers.Dense(10))
```

`Sequential` groups a linear stack of layers into a `tf.keras.Model`.

2D convolution layer (e.g. spatial convolution over images).

This means the convolutional layer will have a 3×3 kernel (filter) and apply on an input image of 32×32 pixels and 3 channels (the RGB colors). Therefore, the output of this layer will be 32 channels (filters).

`input_shape=(32, 32, 3)` for 32x32 RGB pictures

```
# Define how we are going to compile the model, how we  
# will treat the error and how we will measure the success
```

```
model.compile(optimizer = "adam", loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics = ["accuracy"])
```

```
# Retain the history of training epochs
```

```
history = model.fit(train_images, train_labels, epochs=10, validation_data=(test_images, test_labels))
```

```
# At this point, our training and testing are done, and we are ready to plot and report both of these phases:
```

```
plt.plot(history.history["accuracy"], label = "accuracy")  
plt.plot(history.history["val_accuracy"], label = "val_accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.ylim([0.5, 1])  
plt.legend(loc = "lower right")  
plt.show()
```

Keras Dense layer is the layer (hidden) that contains all the neurons (e.g., 64) that are deeply connected within themselves.

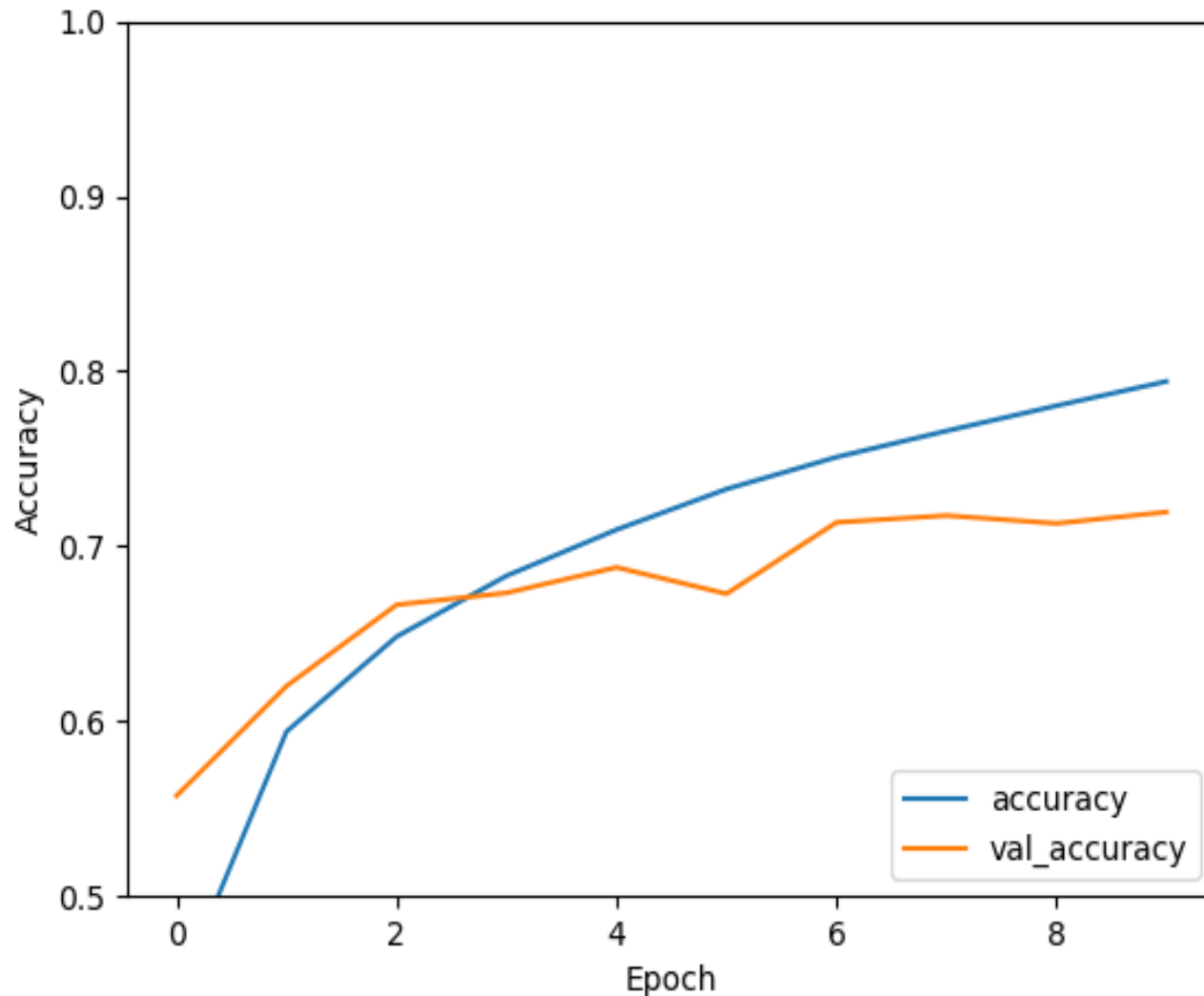
```
# For each of the 10 epochs, along with corresponding accuracy and loss values, we get a plot for training and testing accuracies
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)  
print()  
print("Test Loss: ", test_loss)  
print("Test Accuracy: ", test_acc)
```

activation: Activation function to use. If you don't specify anything, no activation is applied (ie. "linear" activation: $a(x) = x$).

```
if __name__ == "__main__":  
    main()
```

Hands-on Example: Convolution Neural Network



Estimated Time of Arrival (ETA)

Verbose=0 (silent)

Verbose=1 (progress bar)

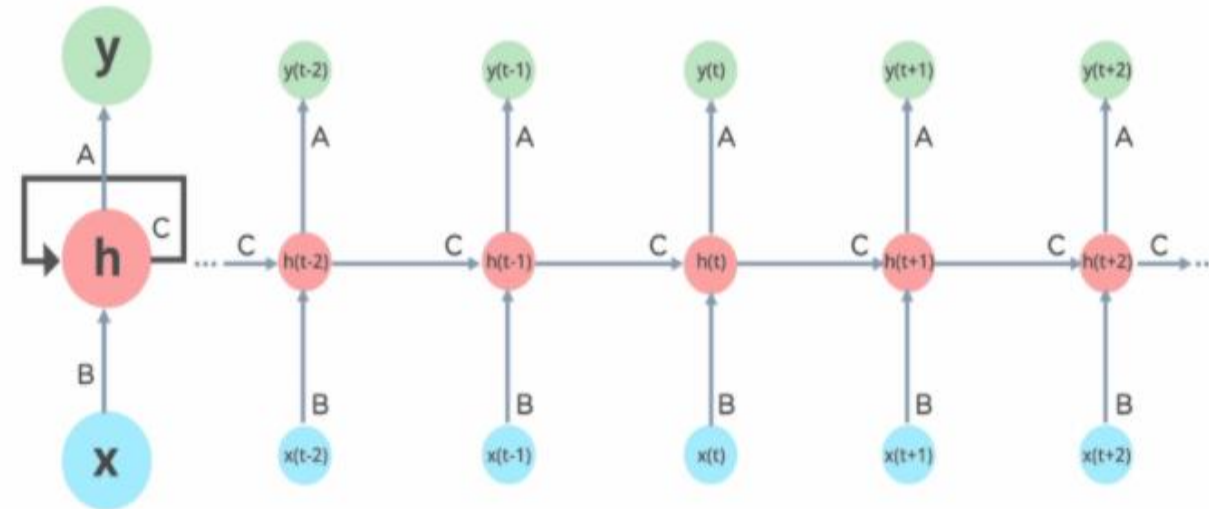
```
Train on 186219 samples, validate on 20691 samples
Epoch 1/2
186219/186219 [=====] - 85s 455us/step - loss: 0.5815 - acc: 0.7728 - val_loss: 0.4917 - val_acc: 0.8029
Train on 186219 samples, validate on 20691 samples
Epoch 2/2
186219/186219 [=====] - 84s 451us/step - loss: 0.4921 - acc: 0.8071 - val_loss: 0.4617 - val_acc: 0.8168
```

Verbose=2 (one line per epoch)

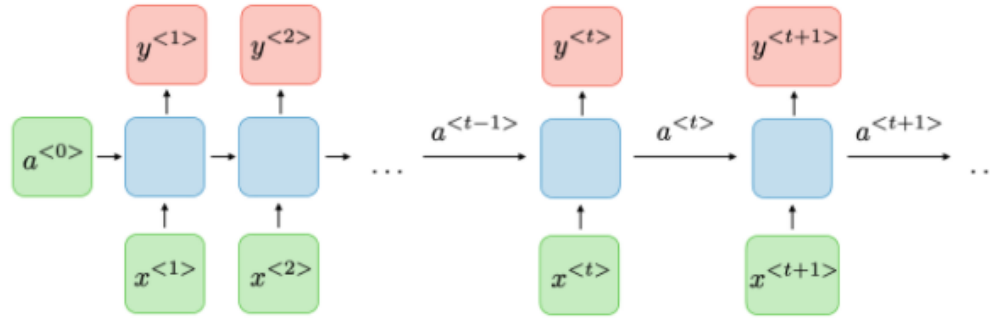
```
Train on 186219 samples, validate on 20691 samples
Epoch 1/1
- 88s - loss: 0.5746 - acc: 0.7753 - val_loss: 0.4816 - val_acc: 0.8075
Train on 186219 samples, validate on 20691 samples
Epoch 1/1
- 88s - loss: 0.4880 - acc: 0.8076 - val_loss: 0.5199 - val_acc: 0.8046
```

Recurrent Neural Networks

- Recurrent neural networks, also known as RNNs, are a class of neural networks that allow previous outputs to be used as inputs while having **hidden states**.
- An RNN is a NN with **a self-loop** in the RNN hidden layer. We unfold the looped RNN with a series of networks that are run over and over again with timed inputs. This is where the “recurrent” name comes from; an RNN does the same task repeatedly, with the output dependent on the previous step in a given time. This allows for what we learn from one pass to influence the next pass.
- RNNs allow cycles in the computational graph.
 - Each cycle has a delay so that each RNN hidden layer may take as an input a value computed from its own output as an earlier step in the computation.
 - The RNN has the **internal state (memory)**: inputs received an earlier time steps affect the RNN’s response to the current input.
- The input layer ‘x’ takes in **the input** to the neural network and processes it and passes it onto the middle layer.
- The middle layer ‘h’ can consist of **multiple hidden layers**, each with its own activation functions and weights and biases.



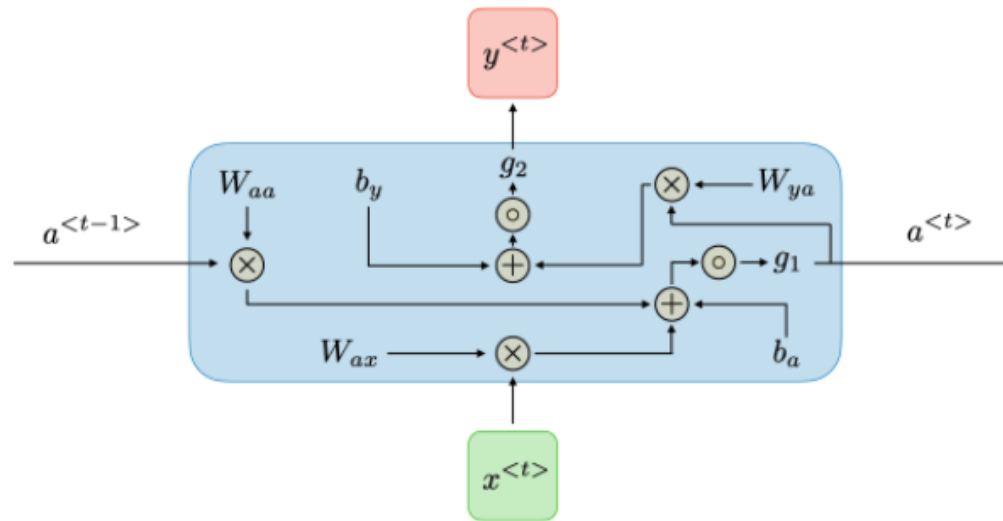
Recurrent Neural Networks



For each timestep t , the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

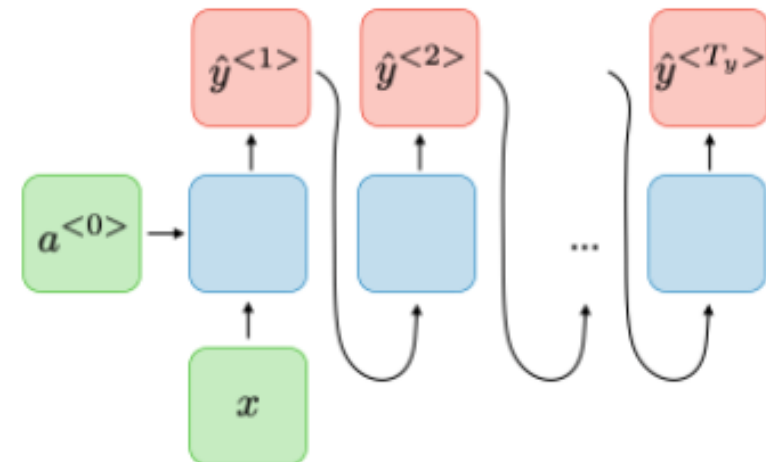
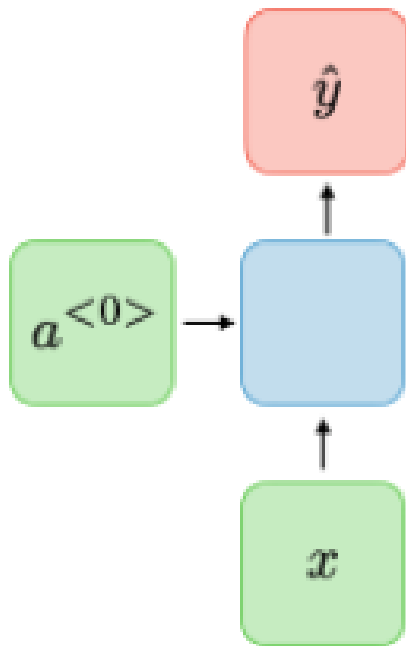
$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \quad \text{and} \quad y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

where W_{ax} , W_{aa} , W_{ya} , b_a , b_y are coefficients that are shared temporally and g_1, g_2 activation functions.



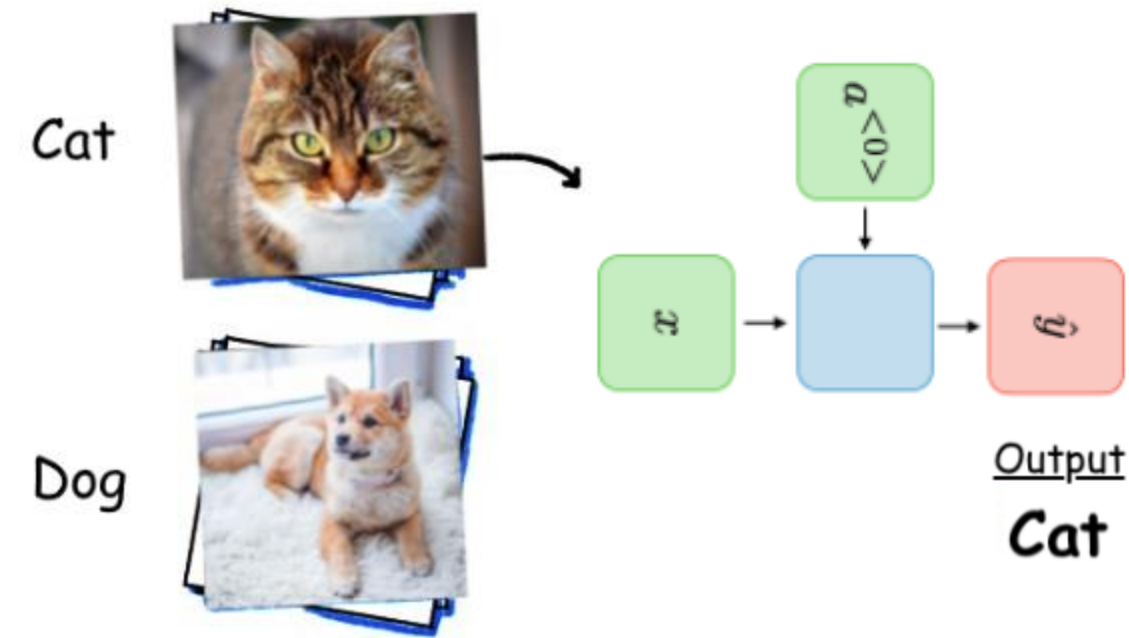
Recurrent Neural Networks

- **One-to-one RNN:** It allows a single input and a single output. It has fixed input and output sizes and acts as a **traditional neural network**.
- An application example can be found in **Image Classification**.
- **One-to-many RNN:** This type of neural network has a single input and multiple outputs.
- An application example can be found in **Image Captioning** that is the process of generating textual description of an image.
- **RNN** is receiving an image representation by a CNN as an input vector $\mathbf{x}$ to generate a sequence of words as output $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$, describing the image.



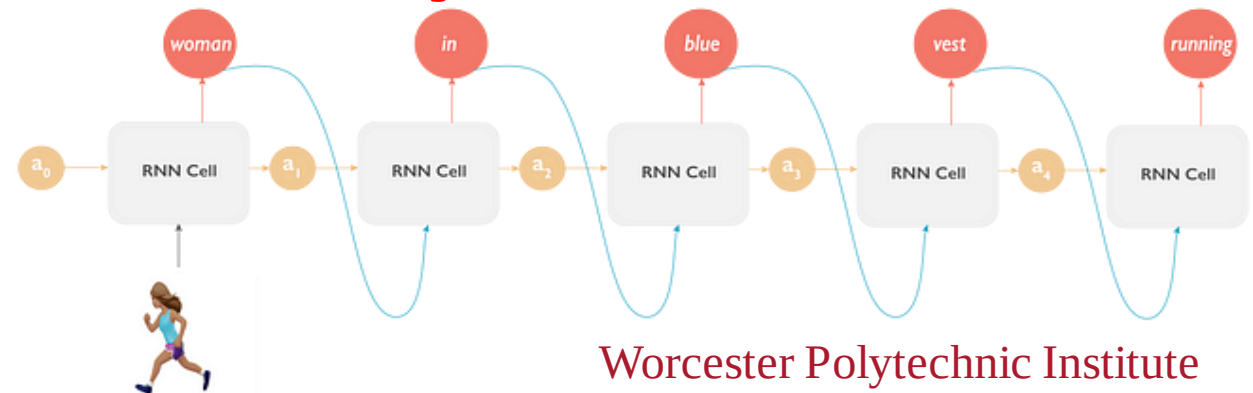
Recurrent Neural Networks

- **One-to-one RNN:** It allows a single input and a single output. It has fixed input and output sizes and acts as a **traditional neural network**.
- An application example can be found in **Image Classification**.
- **One-to-many RNN:** This type of neural network has a single input and multiple outputs.
- An application example can be found in **Image Captioning** that is the process of generating textual description of an image.



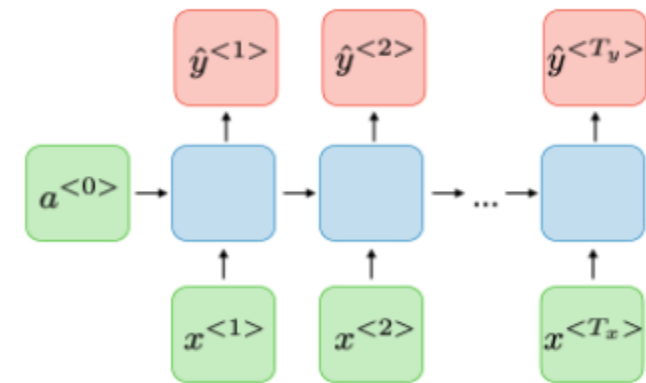
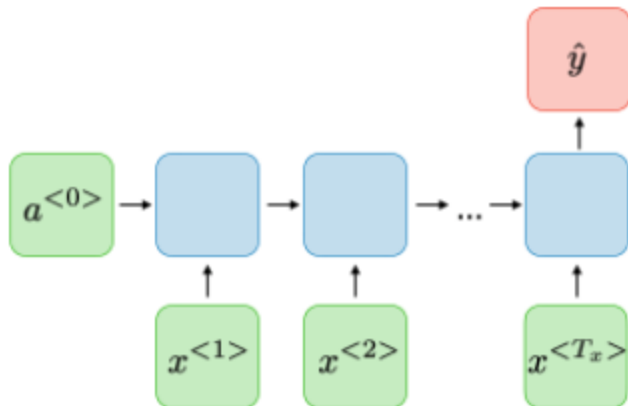
- **RNN** is receiving an image representation by a CNN as an input vector x to generate a sequence of words as output $y_1, y_2, \dots, y_t$, describing the image.

The input is an emoji of a woman running and the output is a sequence of predicted words that describe the image "woman in blue vest running".



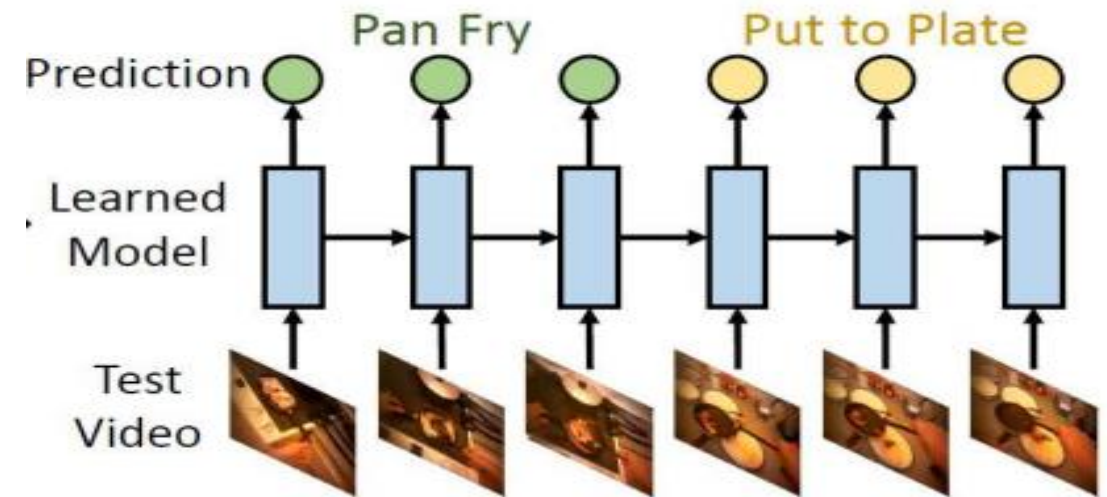
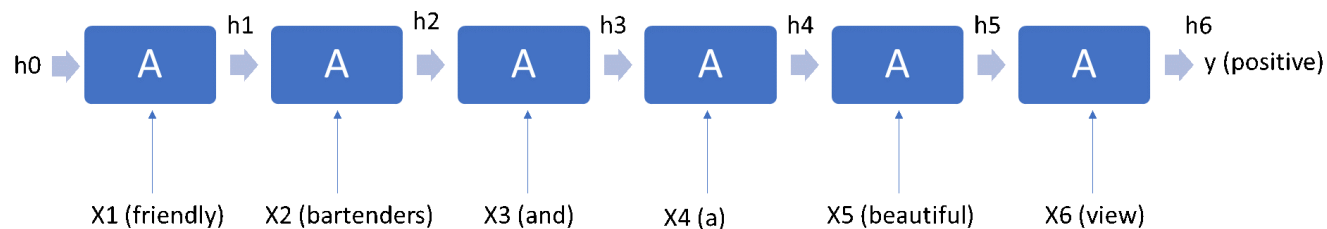
Recurrent Neural Networks

- **Many-to-one RNN:** This RNN takes a sequence of inputs and generates a single output.
- **Sentiment Classification** is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.
- **RNN** is taking a sequence of word representations $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ as input for computing a number $\mathbf{y}$ which denotes the sentiment of the input sentence.
- **Many-to-many RNN:** This RNN takes a sequence of inputs and generates a sequence of outputs.
- **Video Action Classification** is an example to map a sequence, which is the representations of video frames ($\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$) given by a CNN, to a sequence of classes ($\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$), which is used for classifying the action in the video.



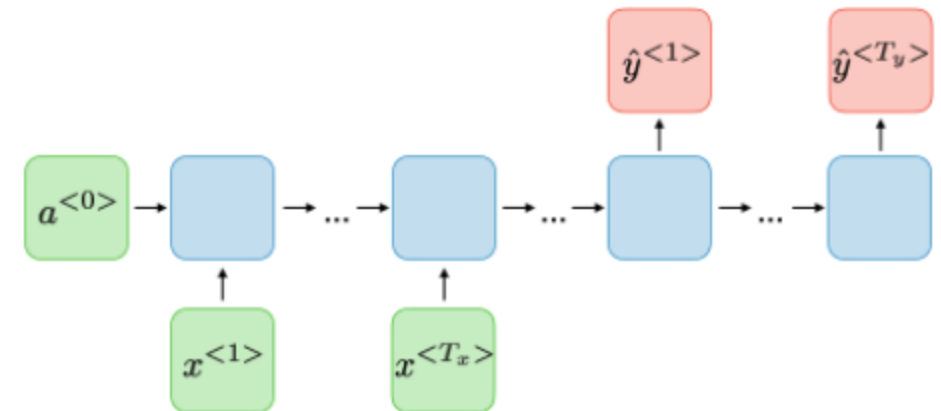
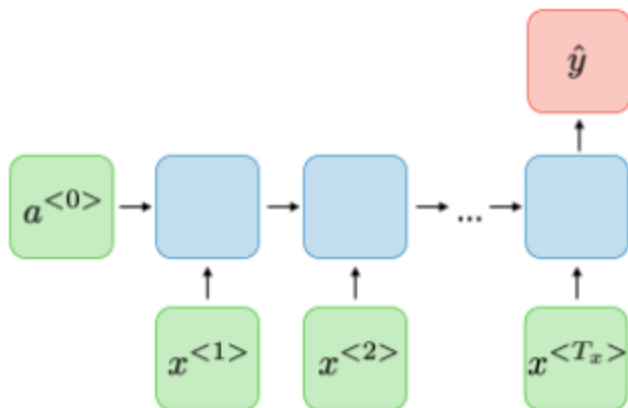
Recurrent Neural Networks

- **Many-to-one RNN:** This RNN takes a sequence of inputs and generates a single output.
- **Sentiment Classification** is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.
- **RNN** is taking a sequence of word representations $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ as input for computing a number $\mathbf{y}$ which denotes the sentiment of the input sentence.
- **Many-to-many RNN:** This RNN takes a sequence of inputs and generates a sequence of outputs.
- **Video Action Classification** is an example to map a sequence, which is the representations of video frames ($\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$) given by a CNN, to a sequence of classes ($\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$), which is used for classifying the action in the video.



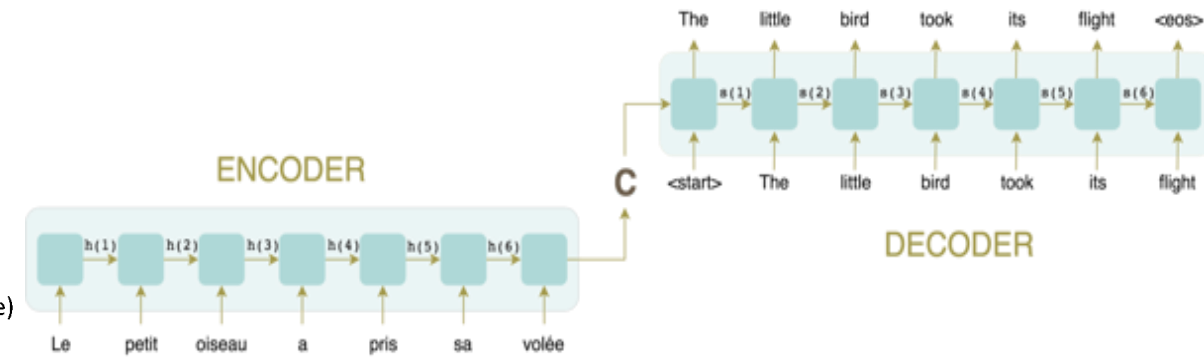
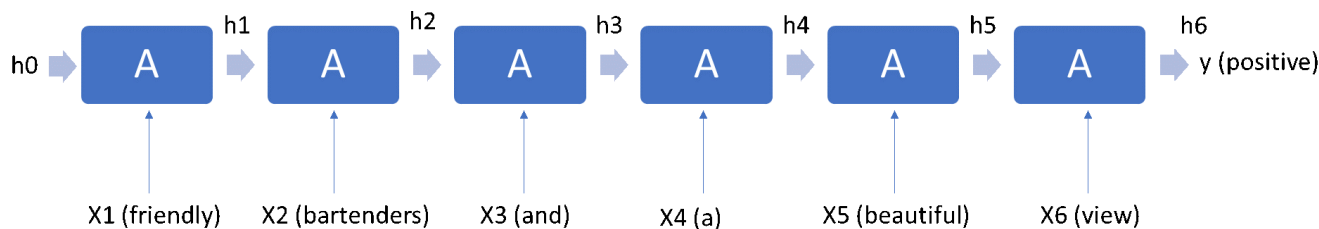
Recurrent Neural Networks

- **Many-to-one RNN:** This RNN takes a sequence of inputs and generates a single output.
- **Sentiment Classification** is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.
- **RNN** is taking a sequence of word representations $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ as input for computing a number $\mathbf{y}$ which denotes the sentiment of the input sentence.
- **Many-to-many RNN:** This RNN takes a sequence of inputs and generates a sequence of outputs.
- **Machine translation** is an example, in which the input sequence that is a sequence in one language is encoded and then decoded to an output sentence in another language.



Recurrent Neural Networks

- **Many-to-one RNN:** This RNN takes a sequence of inputs and generates a single output.
- **Sentiment Classification** is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.
- **RNN** is taking a sequence of word representations $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ as input for computing a number $\mathbf{y}$ which denotes the sentiment of the input sentence.
- **Many-to-many RNN:** This RNN takes a sequence of inputs and generates a sequence of outputs.
- **Machine translation** is an example, in which the input sequence that is a sequence in one language is encoded and then decoded to an output sentence in another language.



CNN vs. RNN

CNN vs. RNN: What are they and how do they differ?

	Convolutional neural network (CNN)	Recurrent neural network (RNN)
ARCHITECTURE	Feed-forward neural networks using filters and pooling	Recurring network that feeds the results back into the network
INPUT/OUTPUT	The size of the input and the resulting output are fixed (i.e., receives images of fixed size and outputs them to the appropriate category along with the confidence level of its prediction)	The size of the input and the resulting output may vary (i.e., receives different text and output translations—the resulting sentences can have more or fewer words)
IDEAL USAGE SCENARIO	Spatial data (such as images)	Temporal/sequential data (such as text or video)
USE CASES	Image recognition and classification, face detection, medical analysis, drug discovery and image analysis	Text translation, natural language processing, language translation, entity extraction, conversational intelligence, sentiment analysis, speech analysis

Hands-on Example: Recurrent Neural Networks

```
import matplotlib.pyplot as plt
import numpy as np
import time
import csv
from keras.layers.core import Dense, Activation, Dropout
import tensorflow as tf
from keras.models import Sequential

bikes = []

def data_bike_num(path_to_dataset='bike_rnn.csv',
                  sequence_length=20,
                  ratio=1.0):
    max_values = ratio * 45949
    with open(path_to_dataset) as f:
        data = csv.reader(f, delimiter=",")
        next(data, None) # skip the headers
        nb_of_values = 0
        for line in data:
            try:
                bikes.append(float(line[0]))
                nb_of_values += 1
            except ValueError:
                pass
            if nb_of_values >= max_values:
                break
    print("Data loaded from csv. Formatting...")
    print(len(bikes))
    print()
```

There are 3 columns in the data. The 1st column is the number of bikes. The 2nd column is the day of the week. The 3rd column is the hour of the day.

Each row represents the number of bikes at a place in each hour.

We are going to build an RNN model with this data and use the weekday and hour information to **predict the number of bikes**. Note that it is a regression problem.

```
result = []
for index in range(len(bikes) - sequence_length):
    result.append(bikes[index: index + sequence_length])
result = np.array(result) # shape (2049230, 50)
result_mean = result.mean()
result -= result_mean
print("Shift: ", result_mean)
print("Data: ", result.shape)
row = int(round(0.95 * result.shape[0]))
train = result[:row, :]
np.random.shuffle(train)
X_train = train[:, :-1]
y_train = train[:, -1]
X_test = result[row:, :-1] # 2297
y_test = result[row:, -1]
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
return [X_train, y_train, X_test, y_test, result_mean]
```

num	weekday	hour
0	5	17
1	5	17
1	5	17
2	5	17
4	5	17
5	5	17
5	5	17
5	5	17
4	5	17
3	5	17

Hands-on Example: Recurrent Neural Networks

```
# Function to build the RNN-based model
# Return sequences refer to return the hidden state <t>.
# By default, the return_sequences is set to False in Keras RNN Layers, and
# this means the RNN layer will only return the last hidden state output <T>.
# The last hidden state output captures an abstract representation of the input sequence.
# When return_sequences parameter is True, it will output all the hidden states of each time steps.
def build_model():
    model = Sequential()
    layers = [1, 50, 100, 1]

    # layers[1]: 50 neurons in this SimpleRNN layer.
    # input_shape=(None, layers[0]): A None dimension means that it can be any scalar number.
    # layers[0] = 1 refers to 1 dimensional input vector.
    # return_sequences=True: It will output all the hidden states of each time steps.

    model.add(tf.keras.layers.SimpleRNN(
        layers[1],
        input_shape=(None, layers[0]),
        return_sequences=True))
    model.add(Dropout(0.2))
    model.add(tf.keras.layers.SimpleRNN(
        layers[2],
        return_sequences=False)) # This means the RNN layer will only return the last hidden state output
    model.add(Dropout(0.2))
    model.add(Dense(layers[3])) # This output layer only has 1 neuron
    model.add(Activation("linear"))
    start = time.time() # time.time() method of Time module is used to get the time in seconds since epoch.
    model.compile(loss="mse", optimizer="rmsprop", metrics=['mae', 'mape'])
    print("Compilation Time : ", time.time() - start)
    return model
```

```
def main():
    epochs = 2
    ratio = 1
    sequence_length = 20
    path_to_dataset = 'bike_rnn.csv'
    X_train, y_train, X_test, y_test, result_mean = data_bike_num(path_to_dataset, sequence_length, ratio)

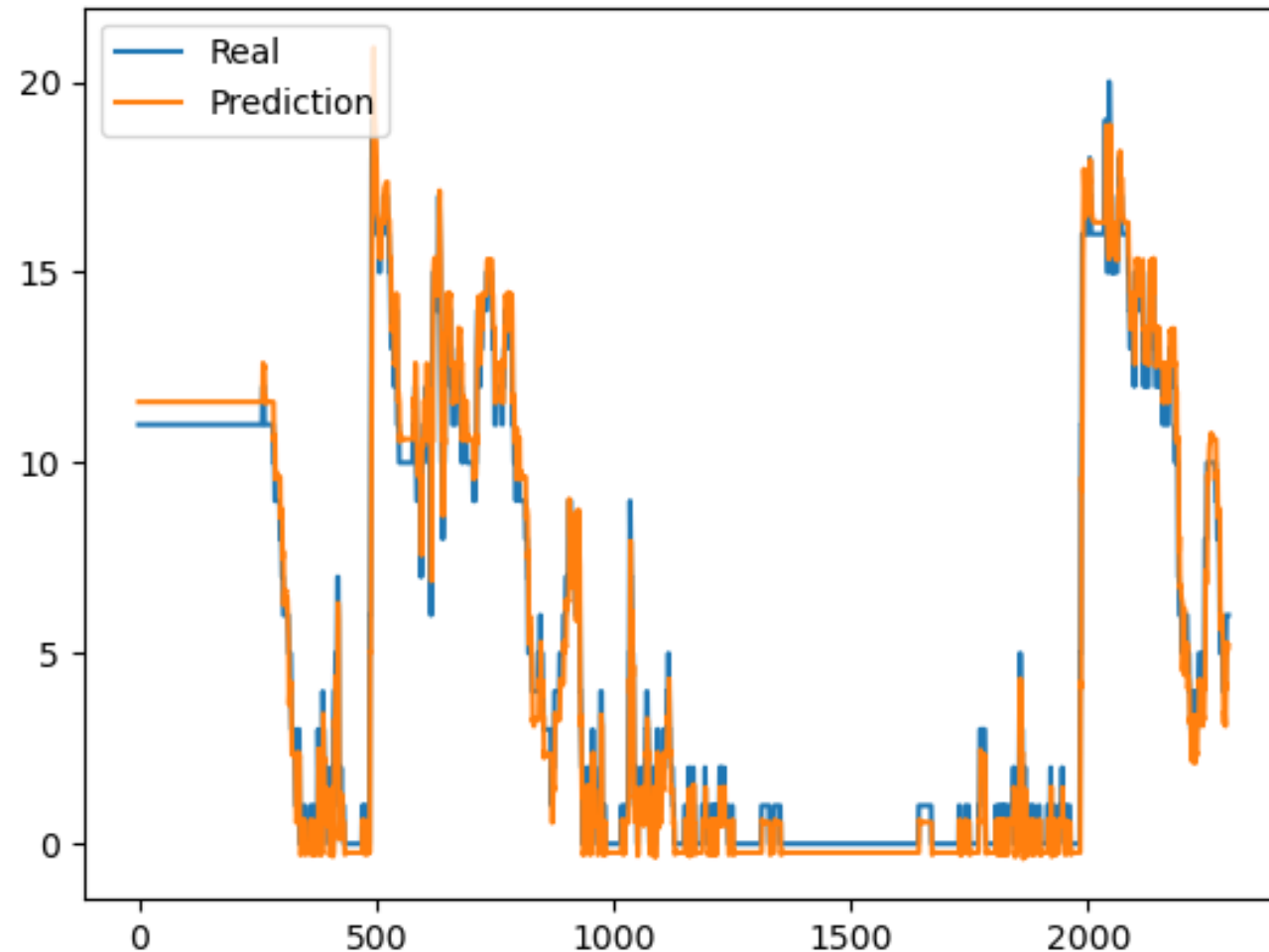
    print('\nData Loaded. Compiling...\n')
    model = build_model()
    model.fit(
        X_train, y_train,
        batch_size=512, epochs=epochs, validation_split=0.05)
    predicted = model.predict(X_test)
    predicted = np.reshape(predicted, (predicted.size,))

    # Evaluate
    # MSE = Mean Squared Error
    # MAE = Mean Absolute Error
    # MAPE = Mean Absolute Percentage Error
    scores = model.evaluate(X_test, y_test, batch_size=512)
    print("\nEvaluation results: \nMSE={:.6f}\nMAE={:.6f}\nMAPE={:.6f}".format(scores[0], scores[1], scores[2]))

    # Draw the figure
    y_test += result_mean
    predicted += result_mean
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.plot(y_test, label="Real")
    ax.legend(loc='upper left')
    plt.plot(predicted, label="Prediction")
    plt.legend(loc='upper left')
    plt.show()

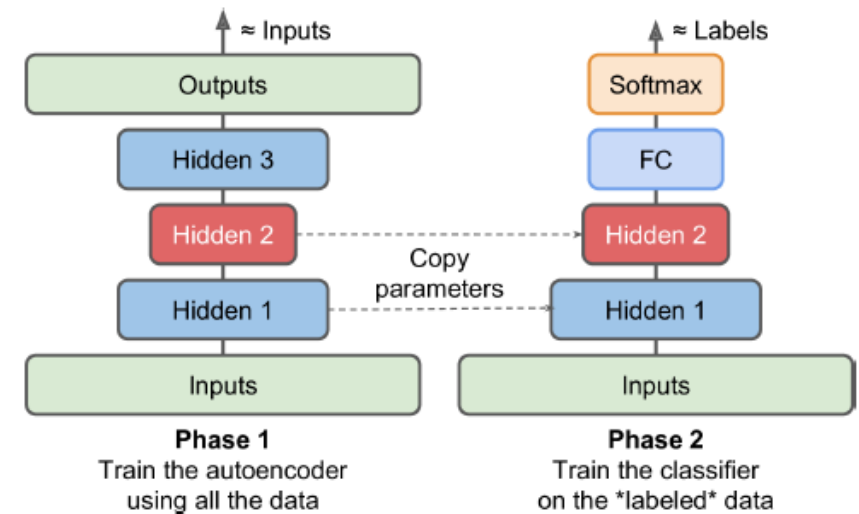
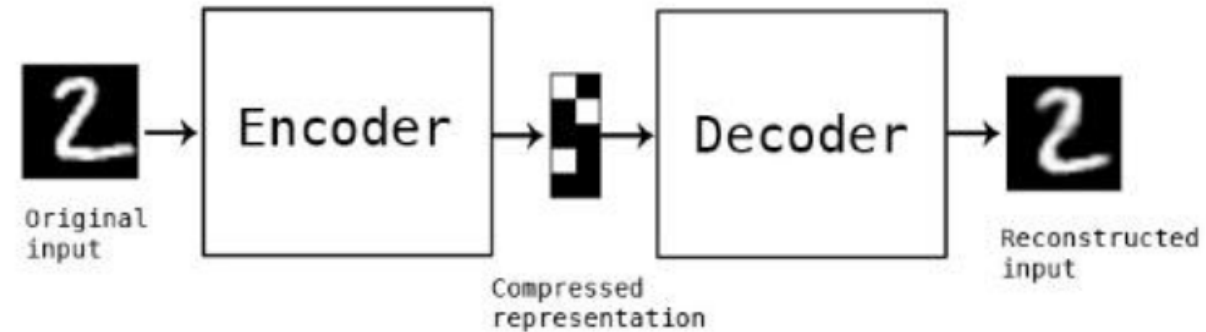
if __name__ == "__main__":
    main()
```

Hands-on Example: Recurrent Neural Networks



AutoEncoder

- A Deep Learning Neural Network Model that can encode the data and learn to simplify the data representation, and then decode it, i.e., making reconstructed data as close as possible to the original input.
- Generally, autoencoders use nonlinear activation functions such as ReLU and sigmoid.
- Why would we want to create a NN that reconstructs the input?
 - Learn the essence of the input, not memorizing it, and then reconstruct it.
 - Let it produce other types of patterns or answers from that learned representation.
- Dimensionality Reduction and Data Compression
- If an autoencoder uses linear activation functions in every layer, the data in the smallest layer in the network is equal to the principal components in the PCA.

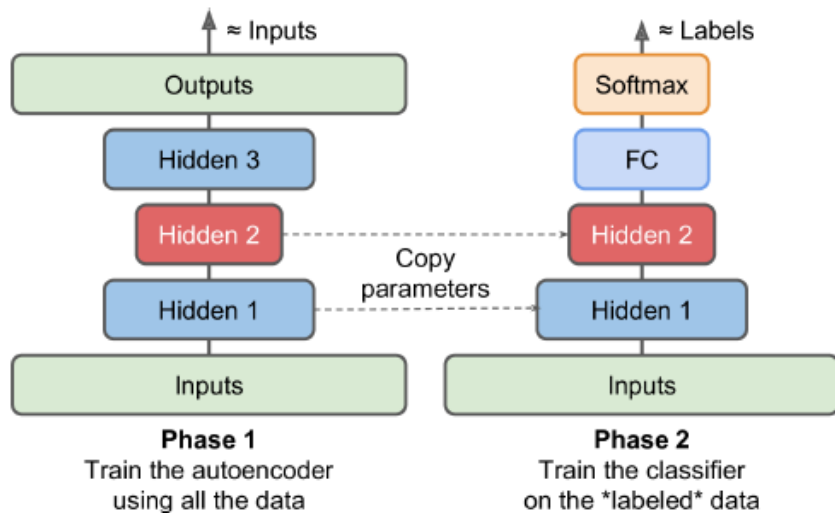
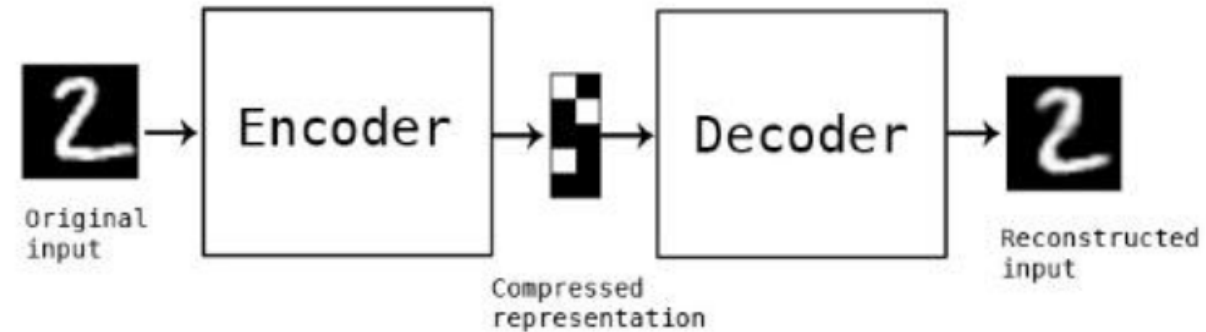


Unsupervised Pre-training

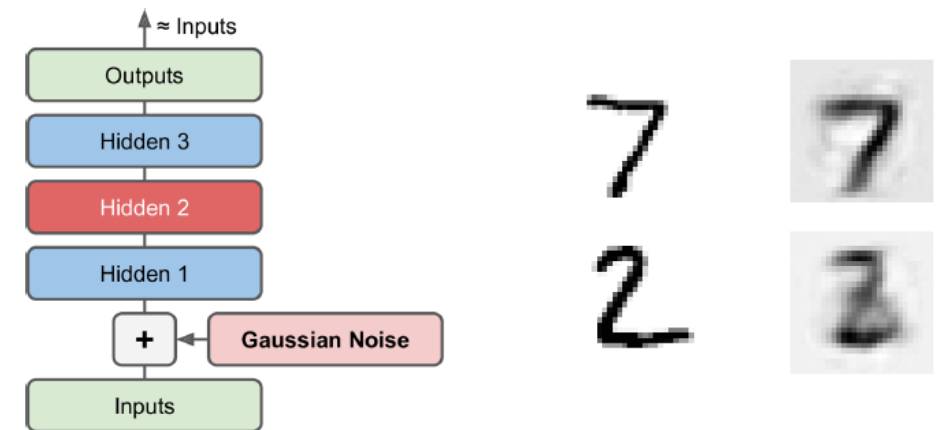
Worcester Polytechnic Institute

AutoEncoder

- Train an autoencoder on pictures of faces to generate new faces. Unsupervised pre-training using stacked autoencoders
- Removing noise from images: An autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output



Unsupervised Pre-training



Stacked denoising autoencoder

Hands-on Example: AutoEncoder

```
import keras
from keras import layers

# This is our dataset
from keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

def main():
    # This is the size of our encoded representations
    encoding_dim = 32

    # This is our input image
    input_img = keras.Input(shape=(784,))

    # "encoded" is the encoded representation of the input
    encoded = layers.Dense(encoding_dim, activation = "relu")(input_img)

    # "decoded" is the lossy reconstruction of the input
    decoded = layers.Dense(784, activation = "sigmoid")(encoded)

    # This model maps an input to its reconstruction
    autoencoder = keras.Model(input_img, decoded)

    # This model maps an input to its encoded representation
    encoder = keras.Model(input_img, encoded)

    # This is our encoded (32-dimensional) input
    encoded_input = keras.Input(shape=(encoding_dim,))

    # Retrieve the last layer of the autoencoder model
    decoder_layer = autoencoder.layers[-1]

    # Create the decoder model
    decoder = keras.Model(encoded_input, decoder_layer(encoded_input))

    # Our network is ready. It is time to set up the other parameters,
    # using the "adam" optimizer and the "binary_crossentropy" as the loss function
    autoencoder.compile(optimizer="adam", loss="binary_crossentropy")

    # Load MNIST dataset and do preprocessing
    (x_train, _), (x_test, _) = mnist.load_data()
    x_train = x_train.astype("float32") / 255.
    x_test = x_test.astype("float32") / 255.
    x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
    x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))

    # Print out the sizes of our training and testing set.
    print(x_train.shape)
    print(x_test.shape)

    # Train the model
    autoencoder.fit(x_train, x_train, epochs=50, batch_size=256,
                    shuffle=True, validation_data=(x_test, x_test))

    # Load up our testing data
    # Encode and decode some digits
    # Note that we take them from the *test* set
    encoded_imgs = encoder.predict(x_test)
    decoded_imgs = decoder.predict(encoded_imgs)
```

Hands-on Example: AutoEncoder

```
# See the reconstruction of our digital images
# Visualizing the digits
n = 10 # How many digits we will display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()

if __name__ == "__main__":
    main()
```



The top is the original image. and the bottom is the image reconstructed by an autoencoder.

Transfer Learning

- Experience with one learning task helps an agent learn better on another task.
- Using the weights learned for Task A to a network will be trained for Task B.
- The weights are then updated by gradient descent using data for Task B.
- Mask R-CNN is a Convolutional Neural Network (CNN) and state-of-the-art in terms of image segmentation.

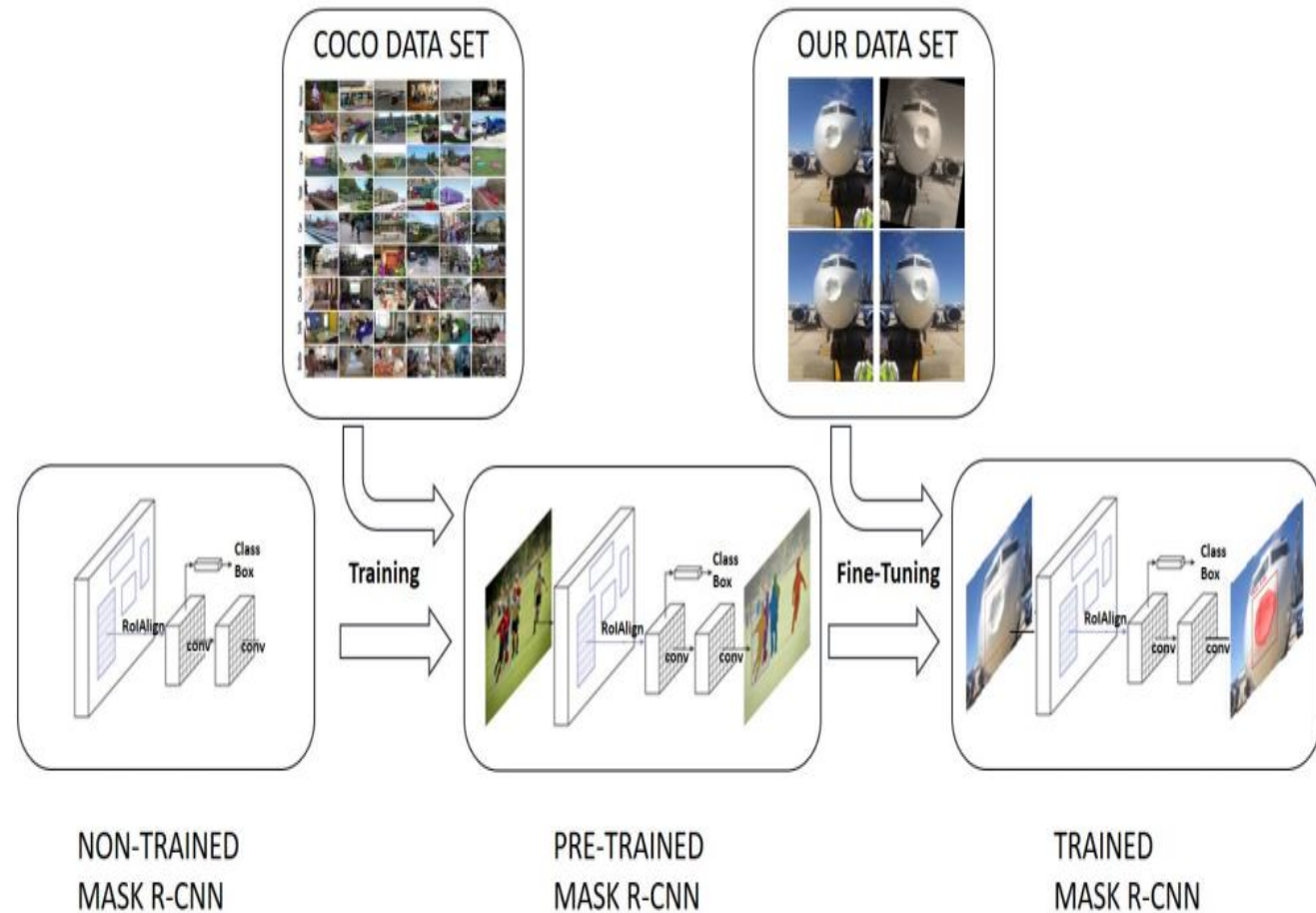
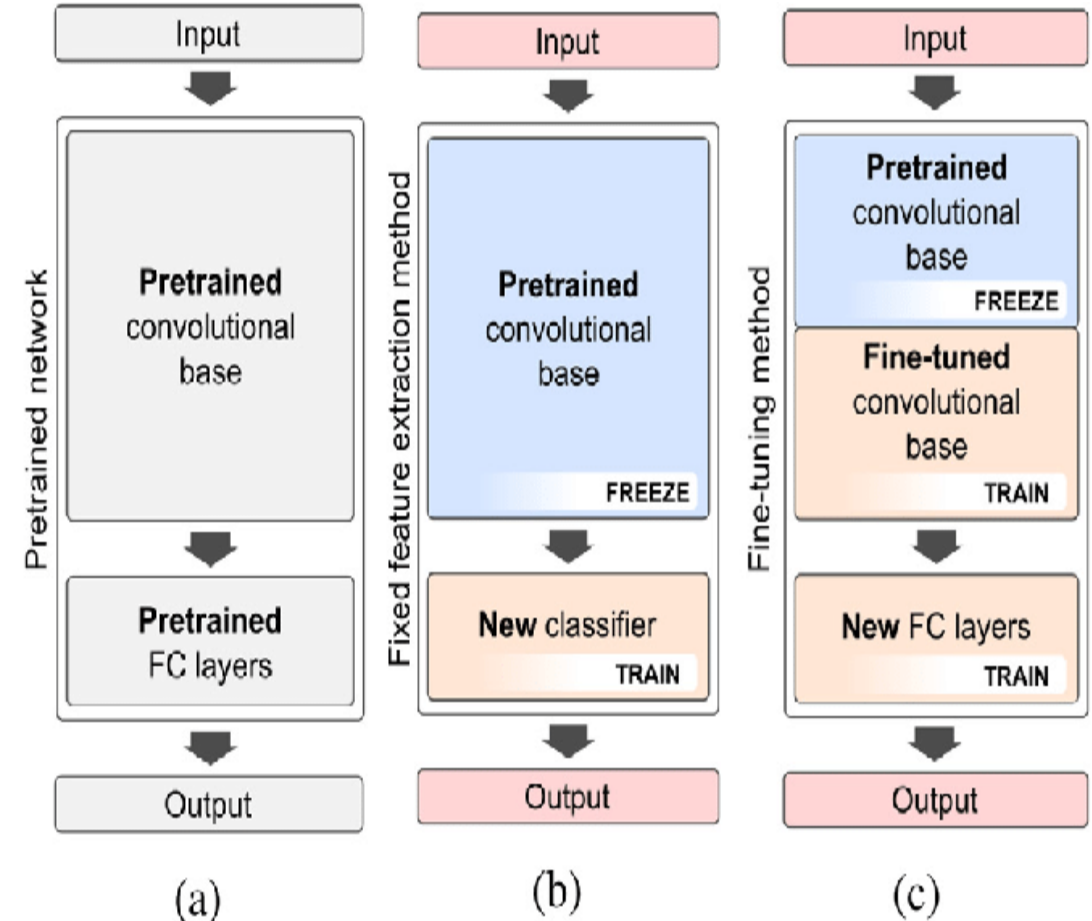


Figure 2. Transfer learning applied in the MASK R-CNN framework.

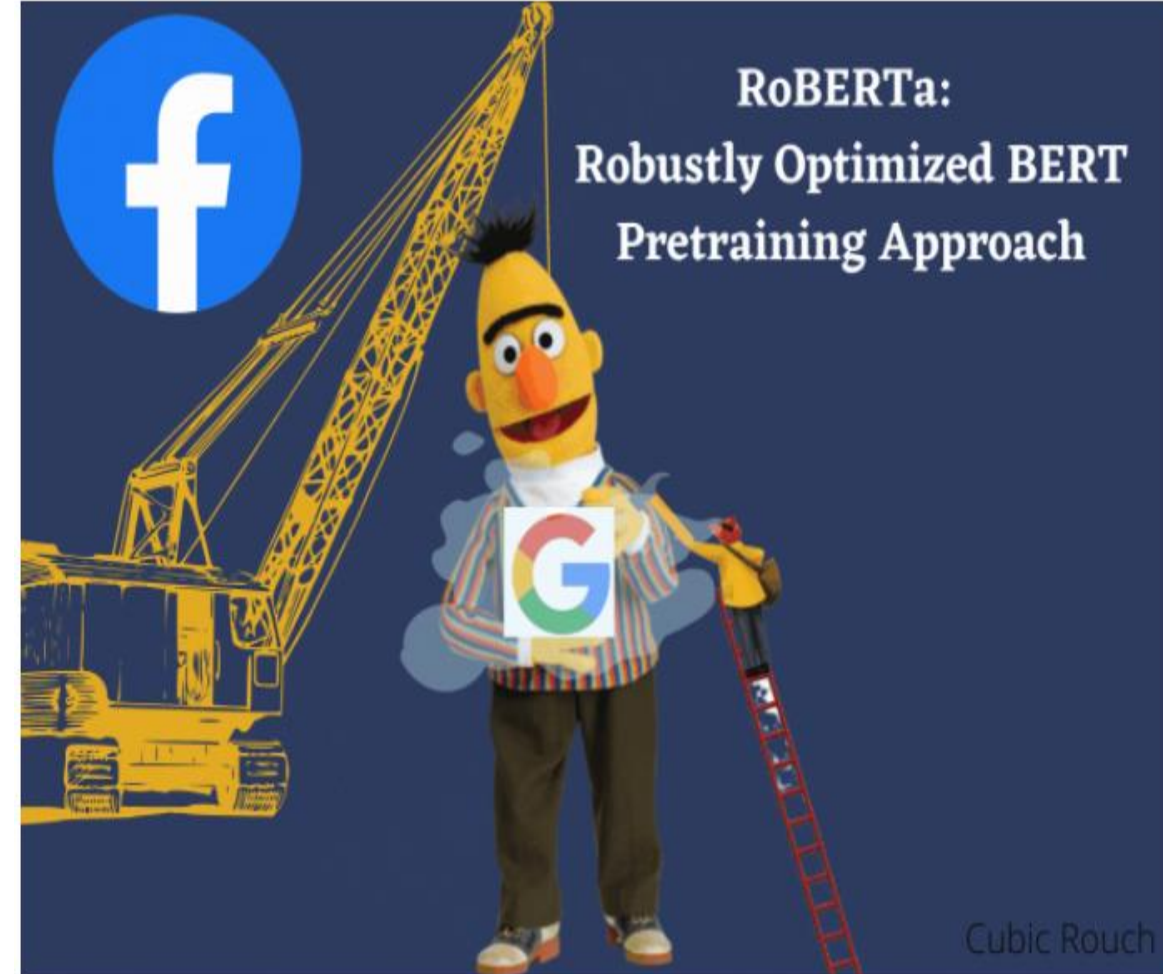
Transfer Learning

- A pre-trained network is a saved network that was previously trained on a large dataset typically on large image classification task
- You can use ImageNet (<https://www.image-net.org/>) and then re-purpose this trained network for something as remote as future items
- Two ways to use pre-trained networks
 - **Feature extraction** – using representations learned by previous network to extract interesting features from new samples
 - **Fine Tuning** – Unfreezing **a few** top layers of convolutional base and jointly training both newly added part of the model and the top layers



Transfer Learning

- The RoBERTa model is a pretrained NLP model.
- By giving it examples of the specialized vocabulary used in the desired domain, e.g., medical domain, financial domain, etc.
- By training the model on the task it is to perform.



Computer Vision

COMPUTER VISION APPLICATIONS



1

ROBOTICS



2

**SELF
DRIVING
CARS**



3

**HEALTH
CARE**



4

MANUFACTURING

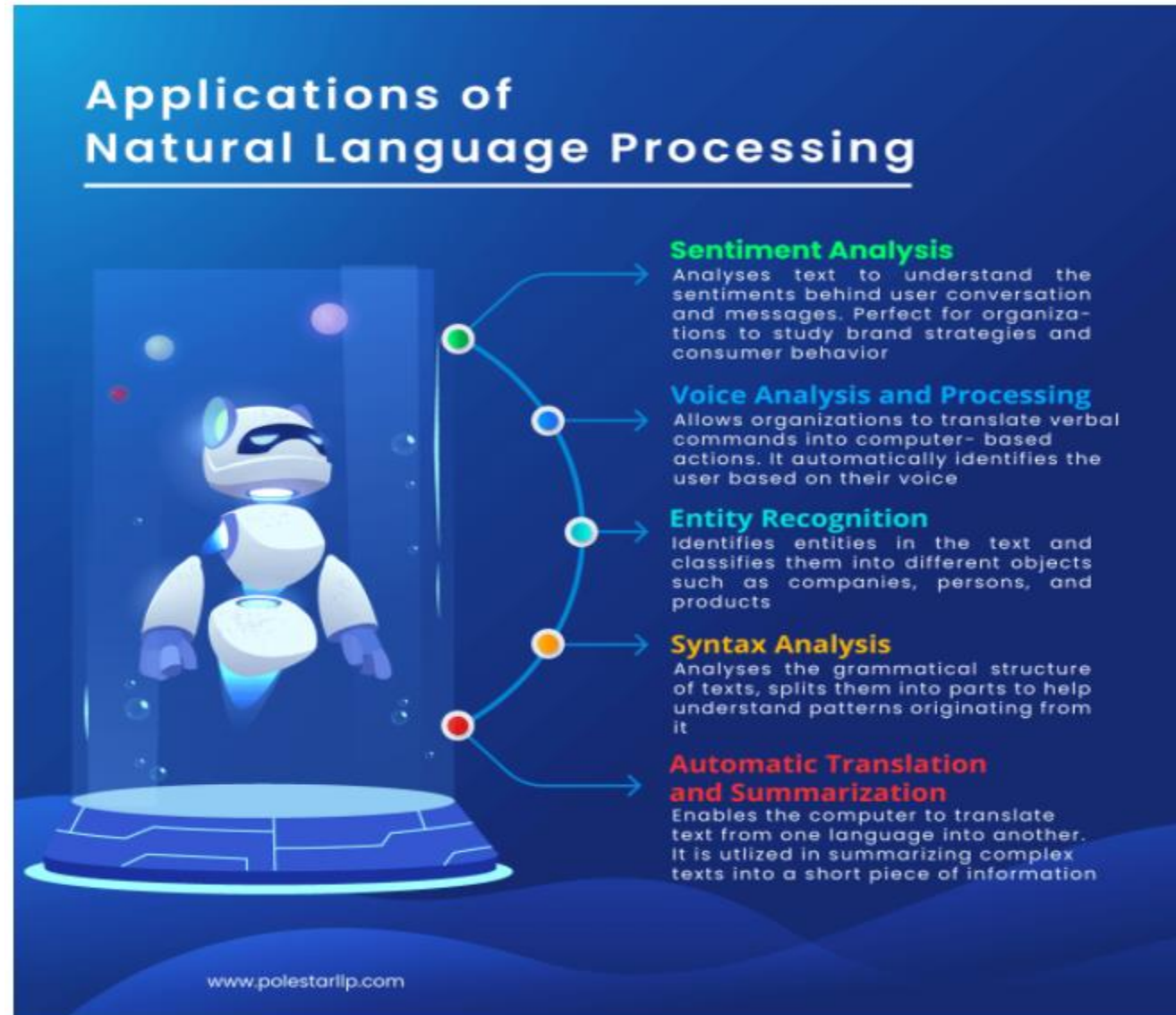


5

AGRICULTURE

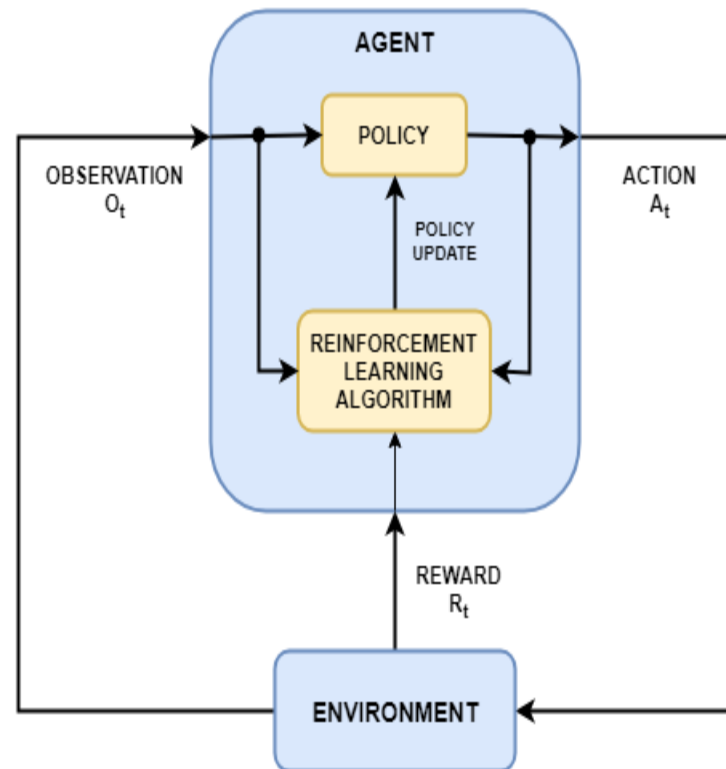
WWW.AIHINTS.COM

Natural Language Processing



Reinforcement Learning

- A decision-making agent learns from a sequence of reward signals that provide some indication of its behavioral quality.
- The goal is to optimize the sum of future rewards.
- The agent contains two components: a policy and a learning algorithm.
 - The **policy** is a mapping that selects actions based on the observations from the environment.
 - The **learning algorithm** continuously updates the policy parameters based on the actions, observations, and reward.
 - The goal of the learning algorithm is to **find an optimal policy** that maximizes the cumulative reward received during the task.



DEEP REINFORCEMENT LEARNING APPLICATIONS

AUTOMOTIVE

A huge dataset that overpowers deep reinforcement learning. It will assist in transforming factories, maintaining vehicles, inclusive automation in the industry.

RESOURCE MANAGEMENT

The use of the reinforcement learning model for businesses constructs deep neural networks. It enables them to learn and divide computer resources towards any pending jobs.

AI TOOLKITS

pre-eminent AI toolkits including OpenAI Gym, Psychlab, and DeepMind Lab. They offer a training environment. It is intrinsic to hurl large-scale innovation for deep reinforcement learning algorithms.

BIDDING & ADVERTISING

RL has proven to be a force in the way marketers bid for ad placements globally. With bidding platforms that bring together a wide range of marketers and business representatives.

MANUFACTURING

Intelligent robots are becoming more commonplace in warehouses and fulfillment centers. To sort out umpteen products along with delivering them to the right people.

HEALTHCARE

Great potential for DRL to advances in healthcare. One of the chief deep RL applications in healthcare includes the diagnosis of diseases, drug manufacturing, and clinical trial & research.

TRAFFIC CONTROL

RL can better create traffic systems that not only provide insight into pre-existing data. Also, help establish trends that can help researchers and city planners better understand their population's behavioral trends.

BOTS

Making AI bots possible leverages the power of deep RL. The bots are learning the semantics and nuances of language in various domains. For both natural language & automated speech understanding!

VIDEO GAMES

Deep RL is using to make complex interactive video games. The RL agent's behavior vicissitudes are based on its learning from the game to optimize the score. It is also used in PC games such as 'Chess' or Atari games.

Computer Framework for Deep Learning Implementation

- H2O. <http://docs.h2o.ai/h2o/latest-stable/h2o-docs/data-science/deep-learning.html>
- Theano. <http://www.deeplearning.net/software/theano/>
- Lasagne. <https://github.com/Lasagne/Lasagne>
- NoLearn. <https://github.com/dnouri/nolearn>
- ConvNetJS. <https://cs.stanford.edu/people/karpathy/convnetjs/index.html>
- Caffe. <https://caffe.berkeleyvision.org/>
- TensorFlow. <https://www.tensorflow.org/>
- Keras. <https://keras.io/>
- Chainer. <https://chainer.org/>
- PyTorch. <https://pytorch.org/>